



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机系统设计实验报告

PA 2 实验报告

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：关浩

2026 年 4 月 29 日

目录

一、 必答题	2
(一) 必答题 1: static & inline 关键字	2
(二) 必答题 2: Makefile 原理	5
二、 文内思考题	11
(一) Q: 堆栈的来源	11
(二) Q: 死循环报错	12
(三) Q: volatile 关键字	12
(四) Q: 多按键同时检测	15
(五) Q: 调色板渐入渐出	15
三、 代码实现	16
(一) TODO: 运行 dummy 程序	16
1. EFLAGS	16
2. RTL (伪) 指令补全	17
3. 注册到 opcode 表	18
4. 实现 helper 函数	20
5. 测试	23
(二) CHECKPOINT: PA2 stage1	24
(三) TODO: 实现更多的指令	25
(四) TODO: 实现 differential testing	33
(五) CHECKPOINT: PA2 stage2	34
(六) TODO: 实现 in/out 指令并运行 Hello World	34
(七) TODO: 实现 ioe (1) - _uptime()	36
(八) TASK: 用时钟测试计算机性能	37
(九) TODO: 实现 ioe (2) - _read_key()	38
(十) TODO: 添加内存映射 I/O	40
(十一) TODO: 实现 ioe (3) - _draw_rect()	41
(十二) TODO: 运行打字小游戏	42
(十三) CHECKPOINT: PA2 stage3	44
四、 实验环境说明	45
(一) 虚拟机配置	45
(二) IDE 配置	45
五、 总结	46

一、 必答题

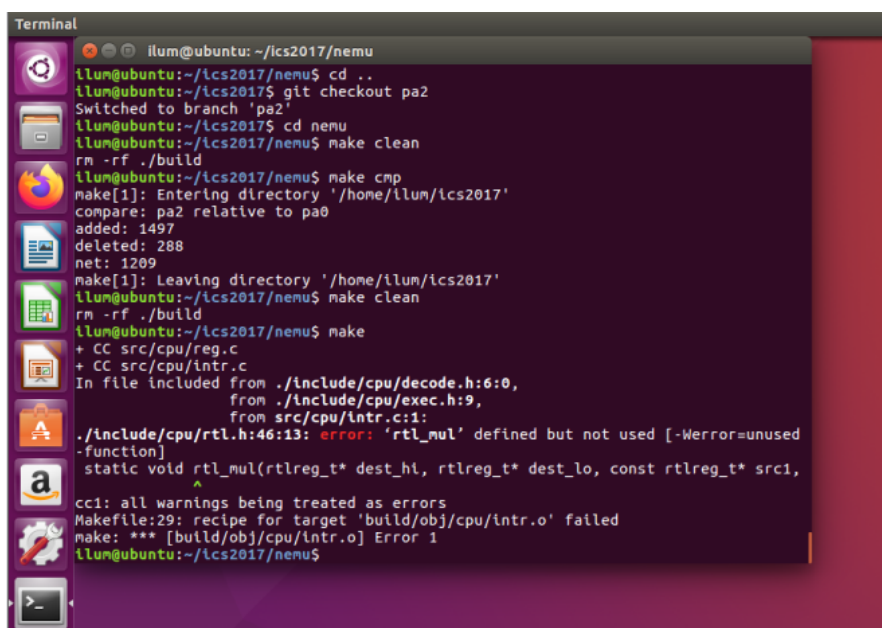
(一) 必答题 1: static & inline 关键字

问: 1. 在 `nemu/include/cpu/rtl.h` 中, 你会看到由 `static inline` 开头定义的各种 RTL 指令函数. 选择其中一个函数, 分别尝试去掉 `static`, 去掉 `inline` 或去掉两者, 然后重新进行编译, 你会看到发生错误. 请分别解释为什么会发生这些错误? 你有办法证明你的想法吗?

答:

选择 `rtl_mul()` 函数。

(1) 单独去掉 `inline`:



```
Terminal
illum@ubuntu: ~/ics2017/nemu
illum@ubuntu:~/ics2017/nemu$ cd ..
illum@ubuntu:~/ics2017$ git checkout pa2
Switched to branch 'pa2'
illum@ubuntu:~/ics2017$ cd nemu
illum@ubuntu:~/ics2017/nemu$ make clean
rm -rf ./build
illum@ubuntu:~/ics2017/nemu$ make cmp
make[1]: Entering directory '/home/illum/ics2017'
compare: pa2 relative to pa0
added: 1497
deleted: 288
net: 1209
make[1]: Leaving directory '/home/illum/ics2017'
illum@ubuntu:~/ics2017/nemu$ make clean
rm -rf ./build
illum@ubuntu:~/ics2017/nemu$ make
+ CC src/cpu/reg.c
+ CC src/cpu/intr.c
In file included from ./include/cpu/decode.h:6:0,
                  from ./include/cpu/exec.h:9,
                  from src/cpu/intr.c:1:
./include/cpu/rtl.h:46:13: error: 'rtl_mul' defined but not used [-Werror=unused-function]
static void rtl_mul(rtlreg_t* dest_hi, rtlreg_t* dest_lo, const rtlreg_t* src1,
                  ^
cc1: all warnings being treated as errors
Makefile:29: recipe for target 'build/obj/cpu/intr.o' failed
make: *** [build/obj/cpu/intr.o] Error 1
illum@ubuntu:~/ics2017/nemu$
```

图 1: ++ static - - inline: compiling error

报错信息: `rtl_mul` 函数定义未使用。

报错原因:

首先 `rtl.h` 会被很多 `.c` 文件包含。 `static` 表示 `rtl_mul` 是内部链接, 每个 `.c` 文件各自拥有一份私有的 `rtl_mul`, 所以不会链接冲突。

但是, 并不是每个包含 `rtl.h` 的 `.c` 文件都会用到 `rtl_mul`。 比如 `intr.c` 包含了 `rtl.h`, 自己有 `rtl_mul` 的定义, 但没有调用它。 “函数定义但未被使用” 本身就是一个编译器会报出的警告/warning, 而现在 Makefile 的编译选项有 `-Werror` 会把警告报成错误/error, 才会出现上面的报错信息 (而不仅仅是警告)。

而如果添加了 `inline`, `static + inline` 的关键字组合本身就有个典型用途: 把小型辅助函数定义在头文件里, 供很多 `.c` 文件按需使用。 gcc 对于其识别, 默认的就不会报出警告 (这是 gcc 本身机制的设计), 而是当作正确的写法——即使被包含在很多 `.c` 文件且并不是都被使用。 所以如果 `static` 和 `inline` 同时存在就会正常编译。

(1) 单独去掉 `static`: 不报错

原因：

`inline` 函数本身主要是提示编译器做内联展开使用的。它不像普通外部函数那样一定要在当前.o 里生成一个全局函数定义。所以虽然 `rtl.h` 被很多.c 包含，但编译器不一定会给每个.o 都生成一个可链接的 `rtl_mul` 全局符号，因此不会报 `multiple definition` 等错误。

但这也是不稳定的，编译器版本不同、C 标准不同、优化级别不同、是否取函数地址、是否真的内联成功，都可能改变最终是否需要一个外部的 `rtl_mul` 定义。所以还是需要加上 `static` 来保证不会产生外部链接冲突。

(3) 两者都去掉：

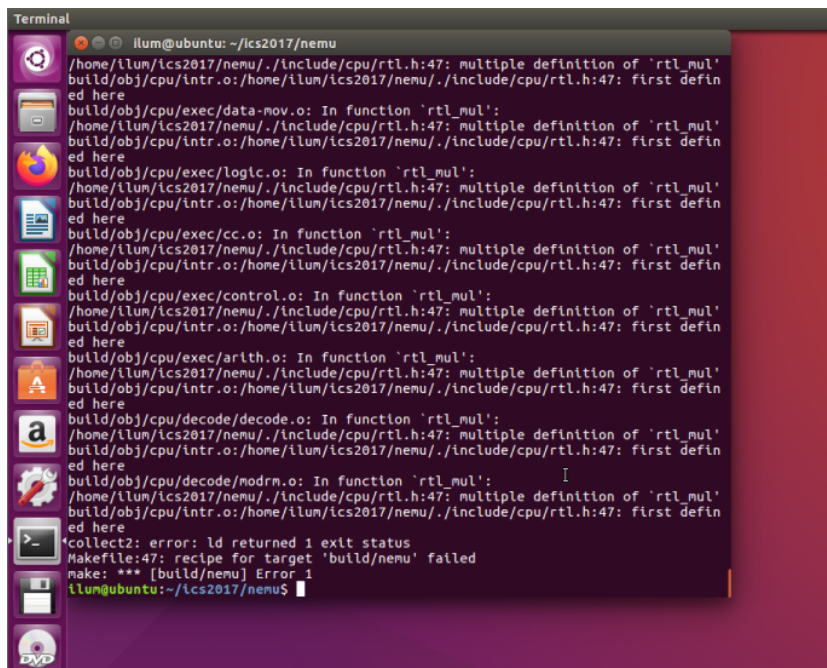


图 2: - - static - - inline: linking error

报错信息：在链接阶段，几乎所有用到 `rtl.h` 的.c 所对应的.o 目标文件都报错，`rtl_mul` 被重复定义。

报错原因：

`rtl_mul` 的函数定义在头文件 `rtl.h` 里。每个包含它的.c 文件，预处理后都会得到一份完整的 `rtl_mul` 函数定义（完整代码）。由于没有 `static`，每个编译产生的.o 文件中都有一个 `rtl_mul` 全局符号。

编译单个.c 时，不会出现问题，因为每个.c 单独看都合法；到了链接阶段，链接器要把所有.o 合成一个 `nemu`，这时就会发现全局符号 `rtl_mul` 有很多份定义，从而对于所有有全局符号的.o 文件，都会报“multiple definition”的错。

当然，如果要定义普通外部函数（无 `static` 和 `inline` 关键字），应该在头文件里只放声明，然后在某一个.c 文件里写唯一的函数定义。

对于后两种情况，即去掉 `static` 的前提下，有/没有 `inline` 关键字、前者不报错后者报错的问题分析，可以通过在.o 文件里查看 `rtl_mul` 的符号来验证。

随意选择两个可能用到 `rtl_mul` 的.c，去它们对应的.o 文件里面找 `rtl_mul`。编译到完成/报错处，然后分别运行以下命令：

```
nm build/obj/cpu/intr.o | grep \texttt{rtl\_mul}
nm build/obj/cpu/exec/arith.o | grep \texttt{rtl\_mul}
```

只去掉 static 保留 inline:

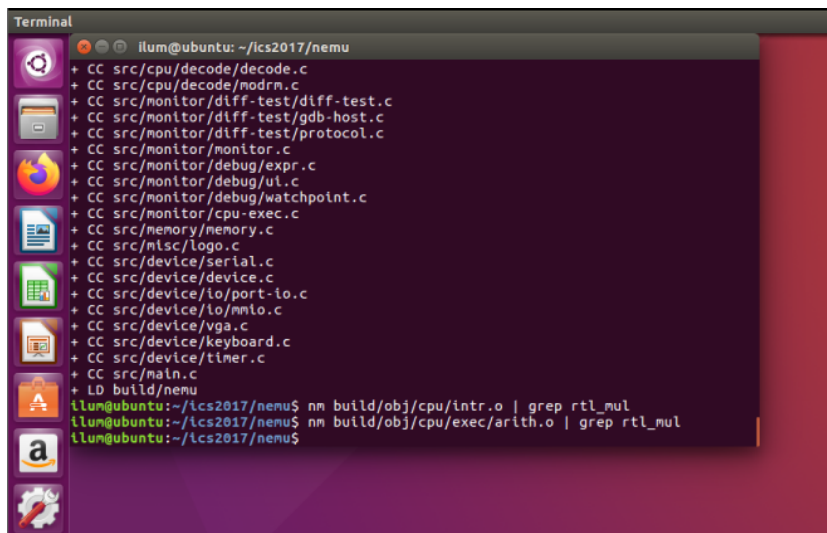


图 3: --static ++ inline: nm 结果

但当两个都去掉:

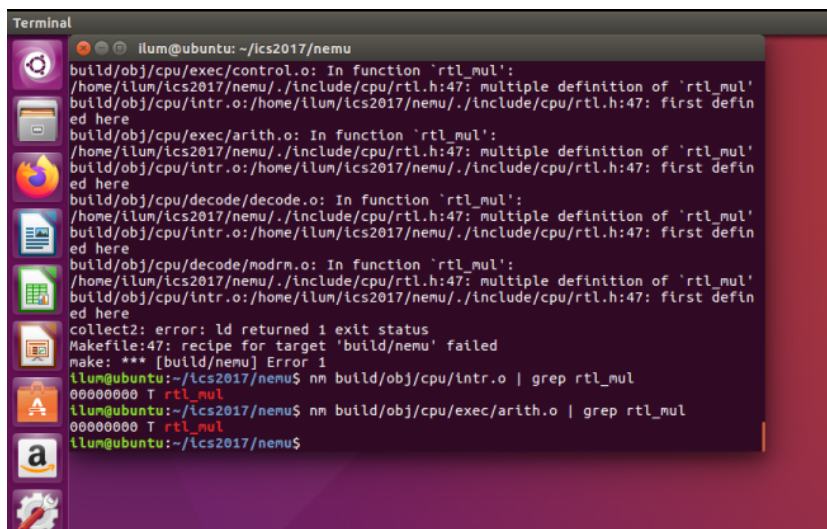


图 4: --static -- inline: nm 结果

从测试结果可以看到, 只去掉 static, 编译出的.o 文件根本就没有 rtl_mul 符号; 而两个关键字都去掉以后, 编译出的所有相关.o 文件都有 T rtl_mul 符号, “T” 说明这是一个全局符号。因此, 只去掉 static 不会出现重复定义, 而全去掉会因为多个同名全局符号产生重复定义的错误。

实际上, 只保留 inline 的时候, 是受到 inline 关键字保护, 这些.c 文件进行编译的时候才都没有各自产生 rtl_mul 符号, 否则按照之前的分析, 也会产生 “T” 标识的全局符号, 还是会产生重复定义的问题。

(二) 必答题 2: Makefile 原理

问： 了解 Makefile 请描述你在 nemu 目录下敲入 make 后,make 程序如何组织.c 和.h 文件,最终生成可执行文件 nemu/build/nemu.(这个问题包括两个方面:Makefile 的工作方式和编译链接的过程.) 关于 Makefile 工作方式的提示:

- Makefile 中使用了变量, 包含文件等特性
- Makefile 运用并重写了一些 implicit rules
- 在 man make 中搜索-n 选项, 也许会对你有帮助
- RTFM

答：

在 nemu 目录下敲入 make 后, 执行的是 nemu/ 目录下的 Makefile:

```
NAME = nemu
INC_DIR += ./include
BUILD_DIR ?= ./build
OBJ_DIR ?= $(BUILD_DIR)/obj
BINARY ?= $(BUILD_DIR)/$(NAME)

include Makefile.git

.DEFAULT_GOAL = app

TOP := $(shell git rev-parse --show-toplevel 2>/dev/null)
ifeq ($(strip $(TOP)),)
TOP := ..
endif
CMP_ARGS := $(filter-out cmp,$(MAKECMDGOALS))

# Compilation flags
CC = gcc
LD = gcc
INCLUDES = $(addprefix -I, $(INC_DIR))
CFLAGS += -O2 -MMD -Wall -Werror -ggdb $(INCLUDES)

# Files to be compiled
SRCS = $(shell find src/ -name "*.c")
OBJS = $(SRCS:src/%.c=$(OBJ_DIR)/%.o)

# Compilation patterns
$(OBJ_DIR)/%.o: src/%.c
```

```
@echo + CC $<
@mkdir -p $(dir $@)
@$(CC) $(CFLAGS) -c -o $@ $<

# Dependencies
-include $(OBJS:.o=.d)

# Some convinient rules

.PHONY: app run submit clean count cmp
app: $(BINARY)

ARGS ?= -l $(BUILD_DIR)/nemu-log.txt

# Command to execute NEMU
NEMU_EXEC := $(BINARY) $(ARGS)

$(BINARY): $(OBJS)
    $(call git_commit, "compile")
    @echo + LD $@
    @$(LD) -O2 -o $@ $^ -lSDL2 -lreadline

run: $(BINARY)
    $(call git_commit, "run")
    $(NEMU_EXEC)

gdb: $(BINARY)
    $(call git_commit, "gdb")
    gdb -s $(BINARY) --args $(NEMU_EXEC)

clean:
    rm -rf $(BUILD_DIR)

count:
    @$(MAKE) -C $(TOP) count

cmp:
    @$(MAKE) -C $(TOP) cmp $(CMP_ARGS)

ifneq ($(filter cmp,$(MAKECMDGOALS)),)
ifneq ($(CMP_ARGS),)
$(CMP_ARGS):
    @:
```

```
endif
endif
```

这个 Makefile 总体的工作流程是：

.c 文件 → .o 目标文件 → build/nemu 可执行文件
.h 文件 → 作为依赖参与判断是否需要重新编译

下面从“Makefile 的工作方式编译链接的过程”分别分析：

(1) Makefile 的工作方式

从 Makefile 内容逐步分析工作方式，只分析最重要的、直接关系到其工作流程的部分。

[pt.1]

Makefile 的开头定义了一些变量：

```
NAME = nemu
INC_DIR += ./include
BUILD_DIR ?= ./build
OBJ_DIR ?= $(BUILD_DIR)/obj
BINARY ?= $(BUILD_DIR)/$(NAME)
```

可以获取到的信息是：编译产生的可执行文件名为“nemu”；使用头文件的目录是当前目录下的 include 文件夹；使用的编译目录是当前目录下的 build 文件夹；在编译目录 build 下，再创建一个 obj 文件夹用于输出可连接文件；最终生成的二进制可执行文件也放在编译目录 build 下。

——所以编译得到的最终目标文件是 build 文件夹下的 nemu 文件。这一段 Makefile 对编译过程中所生成的文件进行了路径划分。

[pt.2]

```
include Makefile.git
```

Makefile.git 里定义了 git_commit 这个宏，使得 Makefile 指导的编译、运行过程中，能够自动提交 git 记录。

[pt.3]

```
.DEFAULT_GOAL = app
```

以及后续的：


```
app: $(BINARY)
```

这设置了 Makefile 的默认目标是“app”，那么运行 `make` 就等价于运行 `make app`。而 `app` 指向 `pt.1` 中声明的 `BINARY` 路径，也就是最终执行文件，这就是为什么 `make` 会尝试生成最终的 `nemu` 可执行文件。

[pt.4]

```
SRCS = $(shell find src/ -name "*.c")
```

字面语义是：递归找出 `src/` 下所有 `.c` 文件。通过这一句，程序收集编译 `nemu` 所需要的所有的 `.c` 源文件。

```
OBJS = $(SRCS:src/%.c=$(OBJ_DIR)/%.o)
```

再通过这一句将所有收集到的源文件映射成 `.o` 目标文件。这其中涉及到的路径关联关系，可概括如下：

```
src/...(若干路径).../xxx.c -> build/obj/...(同样的若干路径).../xxx.o
```

以上足以解释 Makefile 的工作方式，包括了它如何收集编译所需要的文件、如何决定生成文件的路径、以及如何接受或接受什么样的编译命令。

(2) 编译过程

对于编译，Makefile 使用了模式规则：

```
$(OBJ_DIR)/%.o: src/%.c
    @echo + CC $<
    @mkdir -p $(dir $@)
    @$ (CC) $(CFLAGS) -c -o $@ $<
```

相当于重写了 `make` 默认的 `.c -> .o` 隐式规则。其中 `$@` 表示目标文件（`.o`）；`$<` 表示第一个依赖文件（源文件 `.c`）。其中 `CC` 和 `CFLAGS` 在前文已经被定义：

```
CC = gcc
INCLUDES = $(addprefix -I, $(INC_DIR))
CFLAGS += -O2 -MMD -Wall -Werror -ggdb $(INCLUDES)
```

```
\end{codebox}
```

\texttt{INC_DIR} 在上一部分的分析中提到过，指的是 `./include` 目录。

所以，实际的编译命令形式应是：

```
\begin{codebox}
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/.../xxx.o src/.../xxx.c
```

——xxx 替换为实际的.o/.c 文件名，“...”省略了实际路径。

一些补充说明。`.h` 文件不是直接编译的，而是被.c 文件通过 `include` 引入。`-I./include` 只是告诉 GCC 编译器要去 `./include` 目录下找对应的头文件。

`-O2` 是优化选项；`-Wall -Werror` 在 PA1 已经分析过；`-MMD` 会让 GCC 额外生成.d 依赖文件，记录.o 依赖了哪些.h。而后 Makefile 又通过 `-include $(OBJS:.o=.d)` 把这些.d 文件包含进来。这样如果某个.h 被修改，相关.c 也会重新编译。

以上就是 Makefile 如何指导编译过程的完整分析。

(3) 链接过程

最后再看编译后的链接过程。Makefile 同样编写了如下的模式规则：

```
$(BINARY): $(OBJS)
    $(call git_commit, "compile")
    @echo + LD $@
    @$ (LD) -O2 -o $@ $^ -lSDL2 -lreadline
```

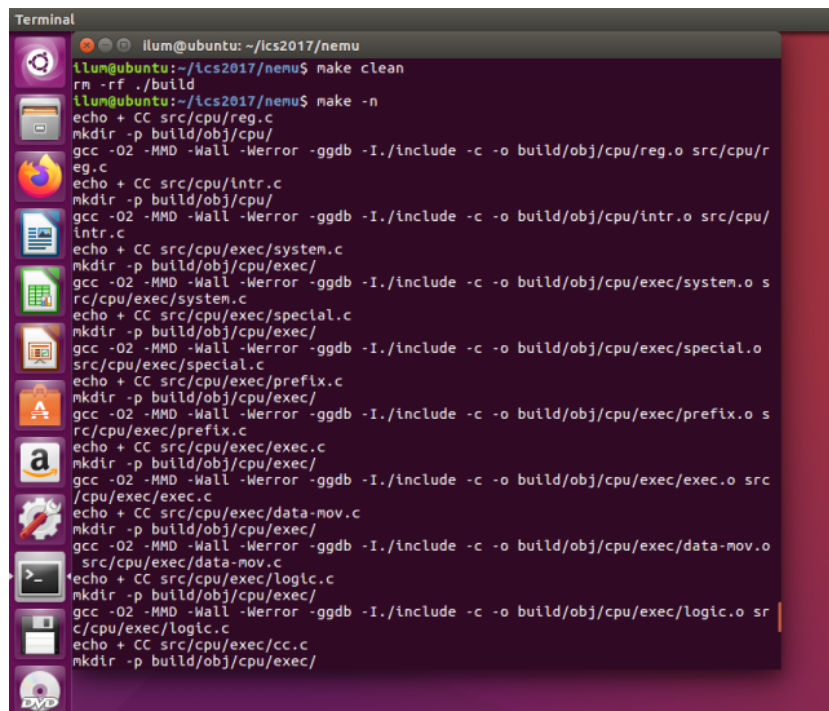
其中 `$@` 表示目标文件，也就是最终生成的 `nemu` 可执行文件；`$` 表示所有依赖，也就是全部目标文件 (.o)。LD 和 CC 一样用的也是 `gcc`，所以实际链接指令大致是：

```
gcc -O2 -o build/nemu build/.../xxx.o build/.../xxx.o ... -lSDL2 -lreadline
```

——中间的 `xxx` 要替换实际目标文件名称，以及补全路径，省略号代表的是所有剩余的相关.o 文件。链接后最终生成的就是 `build` 目录下的 `nemu` 可执行文件，可以直接运行。

(4) 验证

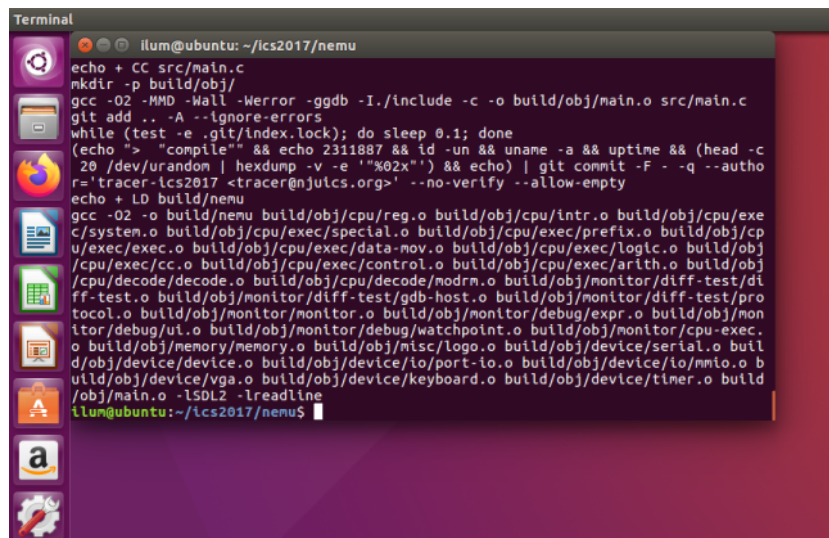
可以通过 `make clean` 后 `make -n`，打印出所有将要执行的命令——受 Makefile 指示的：



```
Terminal
illum@ubuntu: ~/lcs2017/nemu
illum@ubuntu:~/lcs2017/nemu$ make clean
rm -rf ./build
illum@ubuntu:~/lcs2017/nemu$ make -n
echo + CC src/cpu/reg.c
mkdir -p build/obj/cpu/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/reg.o src/cpu/r
eg.c
echo + CC src/cpu/intr.c
mkdir -p build/obj/cpu/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/intr.o src/cpu/
intr.c
echo + CC src/cpu/exec/system.c
mkdir -p build/obj/cpu/exec/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/exec/system.o s
rc/cpu/exec/system.c
echo + CC src/cpu/exec/special.c
mkdir -p build/obj/cpu/exec/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/exec/special.o
src/cpu/exec/special.c
echo + CC src/cpu/exec/prefix.c
mkdir -p build/obj/cpu/exec/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/exec/prefix.o s
rc/cpu/exec/prefix.c
echo + CC src/cpu/exec/exec.c
mkdir -p build/obj/cpu/exec/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/exec/exec.o src
/cpu/exec/exec.c
echo + CC src/cpu/exec/data-mov.c
mkdir -p build/obj/cpu/exec/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/exec/data-mov.o
src/cpu/exec/data-mov.c
echo + CC src/cpu/exec/logic.c
mkdir -p build/obj/cpu/exec/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/cpu/exec/logic.o sr
c/cpu/exec/logic.c
echo + CC src/cpu/exec/cc.c
mkdir -p build/obj/cpu/exec/
```

图 5: make 要执行的编译命令

先进行编译，可以看到编译指令和之前推断出来的基本一致，并且每条编译指令之前还会进行 echo 和 mkdir，在 Makefile 中的模式规则中也有体现。



```
Terminal
illum@ubuntu: ~/lcs2017/nemu
echo + CC src/main.c
mkdir -p build/obj/
gcc -O2 -MMD -Wall -Werror -ggdb -I./include -c -o build/obj/main.o src/main.c
git add .. -A --ignore-errors
while (test -e .git/index.lock); do sleep 0.1; done
(echo "> "compile" && echo 2311887 && id -un && uname -a && uptime && (head -c
20 /dev/urandom | hexdump -v -e "%02x") && echo) | git commit -F - -q --autho
r='tracer-lcs2017 <tracer@njutcs.org>' --no-verify --allow-empty
echo + LD build/nemu
gcc -O2 -o build/nemu build/obj/cpu/reg.o build/obj/cpu/intr.o build/obj/cpu/exe
c/system.o build/obj/cpu/exec/special.o build/obj/cpu/exec/prefix.o build/obj/cp
u/exec/exec.o build/obj/cpu/exec/data-mov.o build/obj/cpu/exec/logic.o build/obj
/cpu/exec/cc.o build/obj/cpu/exec/control.o build/obj/cpu/exec/arith.o build/obj
/cpu/decode/decode.o build/obj/cpu/decode/modrm.o build/obj/monitor/diff-test/di
ff-test.o build/obj/monitor/diff-test/gdb-host.o build/obj/monitor/diff-test/pro
tocol.o build/obj/monitor/monitor.o build/obj/monitor/debug/expr.o build/obj/mon
itor/debug/ui.o build/obj/monitor/debug/watchpoint.o build/obj/monitor/cpu-exec.
o build/obj/memory/memory.o build/obj/misc/logo.o build/obj/device/serial.o buil
d/obj/device/device.o build/obj/device/io/port-io.o build/obj/device/io/mmio.o b
uild/obj/device/vga.o build/obj/device/keyboard.o build/obj/device/timer.o build
/obj/main.o -lSDL2 -lreadline
illum@ubuntu:~/lcs2017/nemu$
```

图 6: make 要执行的链接命令

最后进行链接，这一条链接指令包含了所有在编译阶段生成的.o 文件，所以非常长。基本结构和之前分析出来的是一致的。

二、 文内思考题

PA 在编程指导过程中提出的一些问题，单列出来进行系统的回答。

(一) Q：堆栈的来源

问：我们知道代码和数据都在可执行文件里面，但却没有提到堆（heap）和栈（stack）。为什么堆和栈的内容没有放入可执行文件里面？那程序运行时刻用到的堆和栈又是怎么来的？AM 的代码是否能给你带来一些启发？

答：

堆和栈没有放进可执行文件，根本原因是它们的内容在编译/链接时根本还不存在。可执行文件主要保存的是程序的静态初始状态，而堆和栈是运行时状态（局部变量、函数调用帧、动态申请数据等），它们的内容是在运行时由 CPU 指令和分配器逐步写出来的。

对于堆来说，链接文件 loader.ld 中：

```
SECTIONS {  
    . = 0x100000;  
    .....  
    end = .;  
    _end = .;  
    _heap_start = ALIGN(4096);  
    _stack_pointer = 0x7c00;  
    _heap_end = 0x8000000;  
}
```

.text/.rodata/.data/.bss 被链接到从 0x100000 开始的一段内存。程序镜像结束后，_heap_start 被设为 _end 之后按页对齐的位置，_heap_end 被设成 0x8000000。这说明堆不是可执行文件里的一块已经静态存有内容的空间，而是基本程序镜像后面的一段空闲内存区间。

在 trm.c 中：

```
1 extern char _heap_start;  
2 extern char _heap_end;  
3 extern int main();  
4  
5 _Area _heap = {  
6     .start = &_amp;_heap_start,  
7     .end = &_amp;_heap_end,  
8 };
```

起始，AM 只是告诉程序这段地址范围可作为堆使用。真正的分配器之后再从 _heap.start 到 _heap.end 之间切块和分配。

栈的来源也类似。启动代码 start.S 中，通过：

```
mov $_stack_pointer, %esp
call _trm_init           # never return
```

手动地把栈指针 `%esp` 设置到 `_stack_pointer`。从这一刻开始, `call`、`push`、函数局部变量等就会在这片内存上形成栈内容。

(二) Q: 死循环报错

问: 当用户程序陷入死循环时, 让用户程序暂停下来, 并输出相应的提示信息, 你觉得应该如何实现?

答:

如果在真实机器/系统中: 使用时钟, 周期性通知 CPU, 暂停程序、跳到操作系统的中断处理。在内核态, 统计当前程序运行时间, 如果某个用户程序运行太久没有退出, 就尝试暂停或杀掉它, 并输出“程序可能陷入死循环, 已退出”这类报告信息到 `console`。

如果在当前 NEMU 虚拟机中: `cpu_exec()` 中在执行循环内添加类似时钟或计数的机制, 这样即使用户程序陷入了死循环, 模拟器仍然有机会在每条指令之间检查状态, 然后停回 `monitor`, 打印信息。

无论哪种语境下, 一个基本的思路是: 用一个不依赖用户程序主动配合的机制来夺回 CPU 控制权。

(三) Q: volatile 关键字

问: 也许你从来都没听说过 C 语言中有 `volatile` 这个关键字, 但它从 C 语言诞生开始就一直存在。`volatile` 关键字的作用十分特别, 它的作用是避免编译器对相应代码进行优化。你应该动手体会一下 `volatile` 的作用……你或许会感到疑惑, 代码优化不是一件好事情吗? 为什么会有 `volatile` 这种奇葩的存在? 思考一下, 如果代码中的地址 `0x8049000` 最终被映射到一个设备寄存器, 去掉 `volatile` 可能会带来什么问题?

答:

手册提供了如下 C 程序代码供探究:

```
1 void fun(){
2     volatile unsigned char *p = (void *)0x8049000;
3     *p = 0;
4     while(*p != 0xff);
5     *p = 0x33;
6     *p = 0x34;
7     *p = 0x86;
8 }
```

直接编译得到的汇编代码:

```
.file    "fun.c"
.section    .text.unlikely,"ax",@progbits
.LCOLDB0:
    .text
.LHOTB0:
    .p2align 4,,15
    .globl  fun
    .type   fun, @function
fun:
.LFB0:
    .cfi_startproc
    movb    $0, 134516736
    .p2align 4,,10
    .p2align 3
.L2:
    movzbl  134516736, %eax
    cmpb    $-1, %al
    jne     .L2
    movb    $51, 134516736
    movb    $52, 134516736
    movb    $-122, 134516736
    ret
    .cfi_endproc
.LFE0:
    .size   fun, .-fun
    .section    .text.unlikely
.LCOLDE0:
    .text
.LHOTE0:
    .ident   "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.12) 5.4.0 20160609"
    .section    .note.GNU-stack,"",@progbits
```

去掉 volatile 关键字，编译得到的汇编代码：

```
.file    "fun.c"
.section    .text.unlikely,"ax",@progbits
.LCOLDB0:
    .text
.LHOTB0:
    .p2align 4,,15
    .globl  fun
    .type   fun, @function
```

```

fun:
.LFB0:
    .cfi_startproc
    movb    $0, 134516736
.L2:
    jmp     .L2
    .cfi_endproc
.LFE0:
    .size    fun, .-fun
    .section .text.unlikely
.LCOLDE0:
    .text
.LHOTE0:
    .ident   "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.12) 5.4.0 20160609"
    .section .note.GNU-stack,"",@progbits

```

两者的主要差别在于，加入 `volatile` 关键字以后：

```

    movb    $0, 134516736
    .p2align 4,,10
    .p2align 3
.L2:
    movzbl  134516736, %eax
    cmpb    $-1, %al
    jne     .L2
    movb    $51, 134516736
    movb    $52, 134516736
    movb    $-122, 134516736
    ret

```

——这一部分有增加代码。这里编译器每次循环都会重新从 0x8049000 读值：

```

movzbl 134516736, %eax

```

因为源代码中 `*p` 被赋值的是 0x8049000 地址上的值，在加入 `volatile` 关键字以后，编译器认为这个地址上的值可能一直在变化，每次读取可能都不同。虽然中途 `*p` 被（仅一次性地）赋过 0，但编译器保留了它“取地址上值”的语义，地址上的值可能变化，`*p` 的值也可能变化，所以要坚持一直重复读取（而后去作比较）。

但如果把 `volatile` 关键字去除，会发现：

```
movb    $0, 134516736
.L2:
    jmp .L2
```

汇编中，只是将 `*p` 赋为 0，而不认为它被赋予的语义“地址上的值”会变化。再向下推一点，由于编译器认为 `*p` 被赋为 0 后其值不会再改变，所以 `*p != 0xff` 是一个恒成立的式子，`while` 循环会一直进行下去，于是把它“优化”成了一个死循环，把后续代码逻辑也切除了（因为编译器认为会一直循环，顺理成章认为后面是无法到达的代码块）。而如果这个地址上的值真的是在不停改变的（比如寄存器），有可能会变成 0xff，那么去除 `volatile` 编译运行会使得这个程序就无法达到的预设目的了。

这就是为什么 `volatile` 必须存在。优化对普通内存通常是有好处的，但如果 0x8049000 被映射成设备寄存器，情况就不一样了。设备寄存器不是普通变量，它可能由硬件改变。例如键盘、串口、时钟、显存控制器都可能通过某个地址和 CPU 通信。

【查阅】 `volatile` 原始的语义是：当前对象的值可能会在当前代码之外被改变，所以每次访问都必须真的去内存/设备地址读写，不能把它缓存起来，也不能随便删掉、合并这些访问。

（四） Q：多按键同时检测

问：如何检测多个键同时被按下——在游戏中，很多时候需要判断玩家是否同时按下了多个键，例如 RPG 游戏中的八方向行走，格斗游戏中的组合招式等等。根据键盘码的特性，你知道这些功能是如何实现的吗？

答：

键盘只负责报告按下/释放事件，是否同时按下由程序根据历史状态判断。历史状态由程序自己维护一个“按键状态表”来记录，所以同时按下多少个键都可以通过扫描这个状态表来获悉。

（五） Q：调色板渐入渐出

问：90 年代的游戏里，很多渐出渐入效果都是通过调色板实现的，聪明的你知道其中的玄机吗？

答：

不改屏幕上每个像素的索引，只改调色板里的颜色值。变化时，所有用调色板中同一颜色的像素同步变暗。可以最小化参数的改变。

三、 代码实现

本节按照手册指导顺序，根据 TODO 任务切分小节，逐步解析代码实现思路。

(一) TODO: 运行 dummy 程序

首先要补全一些必要设施。

1. EFLAGS

在 reg.h 中，CPU_state 结构体内，添加 sub 等指令需要使用到的 EFLAGS 寄存器布局，按照指导书要求只实现到运行 dummy 程序所需的几个指令所用到的位即可：

```

1  /* PA 2: 当前已实现指令会用到的 EFLAGS 布局。
2  * 各标志位的位置遵循 i386 对 EFLAGS 的定义：
3  * (受实验报告篇幅限制，图略)
4  * eflags 原始视图用于 restart()/diffptest 初始化或复制整个寄存器；
5  * 位域视图用于让 RTL 直接读写单个标志位，避免每条指令手写移位。
6  */
7  union {
8      rtlreg_t eflags;
9      struct {
10         uint32_t CF      : 1;    // 0
11         uint32_t bit1    : 1;    // 1
12         uint32_t bit2_5  : 4;    // 2..5
13         uint32_t ZF      : 1;    // 6
14         uint32_t SF      : 1;    // 7
15         uint32_t bit8    : 1;    // 8
16         uint32_t IF      : 1;    // 9
17         uint32_t bit10   : 1;    // 10
18         uint32_t OF      : 1;    // 11
19         uint32_t bit12_31: 20;   // 12..31
20     };
21 };

```

查阅 i386 manual 等材料后，可以知道 i386 复位状态要求 EFLAGS 的 1 号位恒为 1，在 monitor.c 中用来初始化 CPU 运行状态的 restart() 函数内设置 EFLAGS 寄存器的初始值：

```

1  static inline void restart() {
2      /* Set the initial instruction pointer. */
3      cpu.eip = ENTRY_START;
4
5      /* PA 2: i386 复位状态要求 EFLAGS 的 [1] 位恒为
6       * 1，并清空这里建模的状态标志。
7       * 后续改变标志位的指令只通过 RTL helper 更新相应的位。
8       */
9      cpu.eflags = 0x00000002;
10
11     /* ... unralted codes */

```

2. RTL (伪) 指令补全

接着还需要补全 RTL 指令接口的实现。

首先在 rtl.h 中添加了一个辅助工具函数：

```
1  /* PA 2: 辅助函数
2   * 根据操作数宽度返回一个 C 层面的辅助值,
3   * 服务于后续 RTL 伪指令或执行函数, 只读取操作数宽度内的有效位。
4   */
5  static inline rtlreg_t rtl_mask(int width) {
6      assert(width == 1 || width == 2 || width == 4);
7      return width == 4 ? 0xffffffffu : ((1u << (width * 8)) - 1);
8  }
```

编写方式仿照文件内的其他函数。它能够根据操作数宽度生成一个有效位掩码,在更新 ZF/CF 标志位等操作的时候,使用这个掩码可以只看当前操作数宽度内的位,不让高位脏数据影响结果。

剩余的,原代码框架几乎已经完备地声明了所有需要的指令伪指令函数接口,标注了 TODO() 并且有注释提示数据的处理传递方式,只需要跟着原代码框架和注释思路补全所有 TODO() 即可(可能有少数对于 dummy 来说用不到的,一并实现)。

其中使用 concat() 构造函数名的宏：

```
1  #define make_rtl_setget_eflags(f) \
2      static inline void concat(rtl_set_, f) (const rtlreg_t* src) { \
3      /* 标志位 RTL 辅助函数只把规范化后的 0/1 写入 CPU 位域。 */ \
4      cpu.f = *src & 0x1; \
5      } \
6      static inline void concat(rtl_get_, f) (rtlreg_t* dest) { \
7      *dest = cpu.f; \
8      }
```

它和后面的实例调用：

```
1  make_rtl_setget_eflags(CF)
2  make_rtl_setget_eflags(OF)
3  make_rtl_setget_eflags(ZF)
4  make_rtl_setget_eflags(SF)
```

两者共同作用,可以展开得到这些函数供后续使用(函数名是由 concat() 拼接 rtl_set_ / rtl_get_ 与寄存器名得到的)：

```
1  rtl_set_CF()
2  rtl_get_CF()
3
4  rtl_set_OF()
5  rtl_get_OF()
6
7  rtl_set_ZF()
8  rtl_get_ZF()
9
10 rtl_set_SF()
```

```
11 rtl_get_SF()
```

它们是用来读写之前在 reg.h 里 CPU_state 声明的 EFLAGS 位域的。

在各个 RTL 伪指令函数的实现中，pop 和 push 指令的实现需要一些特别的设计。

对于 push 指令：

```
1 static inline void rtl_push(const rtlreg_t* src1) {
2     // esp <- esp - 4
3     // M[esp] <- src1
4     /* 先保存一份本地副本。这样可以符合 i386 对 PUSH ESP 的规定：
5      * 压栈的是旧 ESP 值，而不是递减后的栈指针。
6      */
7     rtlreg_t val = *src1;
8     rtl_subi(&cpu.esp, &cpu.esp, 4);
9     rtl_sm(&cpu.esp, 4, &val);
10 }
```

首先保存一份要压入的值，避免后面修改 ESP 影响到取值（特别是 push esp 这种情况）；而后让栈指针下移（栈是向低地址增长的），并把值写到新栈顶。这里关键的一步是事先保存一份压入的值，因为 x86 规定 push esp 压入的是“原来的”ESP，而不是减完之后的——如果不先保存，此类操作就会出错。

对于 pop 指令：

```
1 static inline void rtl_pop(rtlreg_t* dest) {
2     // dest <- M[esp]
3     // esp <- esp + 4
4     /* 在更新栈指针和写目标之前，先把栈顶值读到临时变量中。
5      * 这样也能保证 POP ESP 正确：最终 ESP 应该等于弹出的值。
6      */
7     rtlreg_t val;
8     rtl_lm(&val, &cpu.esp, 4);
9     rtl_addi(&cpu.esp, &cpu.esp, 4);
10    rtl_mv(dest, &val);
11 }
```

函数中，先从栈顶读数据，然后栈顶上移（反增长方向，弹出操作），最后把弹出的值写入目标寄存器。这里同样使用了临时变量 val，关键也是要处理 pop esp 这种直接动 ESP 情况。按照规定，最终 ESP 应该等于“弹出来的那个值”，不使用 val 中转直接动栈顶就会导致最终 ESP 值的错误，因为这种情况下 dest 和 esp 是同样的。

除此之外，其他的伪指令函数，都有注释依据，跟着逻辑进行简单的数值运算和取址操作即可补全，不再赘述分析。

3. 注册到 opcode 表

接下来在全局注册好 push、pop、sub、xor、call、ret 这几种缺失的指令。

手册告诉我们，可以参考反汇编代码 txt：

```
/home/ilum/ics2017/nexus-am/tests/cputest/build/dummy-x86-nemu:
file format elf32-i386
```

Disassembly of section .text:

00100000 <_start>:

```
100000:      bd 00 00 00 00      mov     $0x0,%ebp
100005:      bc 00 7c 00 00      mov     $0x7c00,%esp
10000a:      e8 01 00 00 00      call    100010 <_trm_init>
10000f:      90                  nop
```

00100010 <_trm_init>:

```
100010:      55                  push    %ebp
100011:      89 e5                mov     %esp,%ebp
100013:      83 ec 08              sub     $0x8,%esp
100016:      e8 05 00 00 00      call    100020 <main>
10001b:      d6                  (bad)
10001c:      eb fe                jmp     10001c <_trm_init+0xc>
10001e:      66 90                xchg    %ax,%ax
```

00100020 <main>:

```
100020:      55                  push    %ebp
100021:      89 e5                mov     %esp,%ebp
100023:      31 c0                xor     %eax,%eax
100025:      5d                  pop     %ebp
100026:      c3                  ret
```

首先在 all-instr.h 中补充声明所有指令的 helper 函数:

```
1 /* PA 2: dummy 实际执行流需要的额外执行辅助函数 */
2 make_EHelper(push);
3 make_EHelper(pop);
4
5 make_EHelper(sub);
6 make_EHelper(xor);
7
8 make_EHelper(call);
9 make_EHelper(ret);
```

并在 exec.c 中, 按照反汇编程序中给出的 opcode (nexus-am/tests/cputest/build/dummy-x86-nemu.txt), 把这些补充的指令注册表项。对于 push、pop、xor、call、ret, 直接在单字节 opcode_table 中找到相应的 opcode 槽位改成对应的指令即可:

```
1 opcode_entry opcode_table [512] = {
```

```

2  /* ... */
3  /* 0x30 */  EMPTY, IDEX(G2E, xor), EMPTY, EMPTY,  /* 0x31:
   按位异或 XOR Ev,Gv */
4  /* 0x54 */  EMPTY, IDEX(r, push), EMPTY, EMPTY,  /* 0x55: 压入
   EBP */
5  /* 0x58 */  EMPTY, EMPTY, EMPTY, EMPTY,
6  /* 0x5c */  EMPTY, IDEX(r, pop), EMPTY, EMPTY,  /* 0x5d: 弹出到
   EBP */
7  /* ... */
8  /* 0xc0 */  IDEXW(gp2_Ib2E, gp2, 1), IDEX(gp2_Ib2E, gp2), EMPTY,
   EX(ret),  /* 0xc3: 近返回 RET */
9  /* ... */
10 /* 0xe8 */  IDEX(J, call), EMPTY, EMPTY, EMPTY,  /* 0xe8:
   相对近调用 CALL rel32 */
11 /* ... */

```

ret 使用 EX() 填充, 即没有立即数、所以不需要译码; 其他均使用 IDEX() 填充, 即需要译码操作数、宽度由操作数大小决定。

sub 指令特殊, 第一个字节不能直接确定是 sub 指令, 只能确定是 group-1 立即数算术/逻辑指令 (add/or/adc/sbb/and/sub/xor/cmp), 要确定是它还得看后面的 ModR/M 字节的**中间 3 位**, 所以要放入 make_group 得到的二级分发表, 再查这一个 8 项小表来确定。查 i386 manual 可知 sub 对应的 opcode - 0x83, 属于 group1, 所以加入到代码架构中的 gp1; ModR/M 字节是 0xec = 1110 1100b, 取中间 3 位得到 reg/opcode 字段, 即 101b = 5, 所以在 gp1 第 5 位:

```

1  make_group(gp1,
2      EMPTY, EMPTY, EMPTY, EMPTY,
3      EMPTY, EX(sub), EMPTY, EMPTY)

```

这里用 EX() 填充的原因是, opcode_table 中会对其做译码:

```

1  /* 0x80 */  IDEXW(I2E, gp1, 1), IDEX(I2E, gp1), EMPTY, IDEX(SI2E,
   gp1),

```

decode_SI2E 已经把 dest 和 src 准备好了, 进入 gp1 后只需要选择执行哪个 ALU 函数, 所以 group 表里用 EX(sub) 就够。

4. 实现 helper 函数

然后就可以对 push、pop、sub、xor、call、ret 依次实现 helper 函数。实现过程比较简单, 简述一下每个 helper 函数做了什么。

1. push

```

1  // data-mov.c
2  make_EHelper(push) {
3      /* dummy 中的 PUSH r32 使用 opcode 50+r.r
   译码器已经把源寄存器的值
4      * 读入 id_dest->val, 因此执行阶段只需要完成 i386 的栈操作:
5      * ESP -= 4, 然后把源操作数写入 [ESP]。
6      */

```

```

7   rtl_push(&id_dest->val);
8
9   print_asm_template1(push);
10  }

```

通过 `rtl_push(&id_dest->val)` 完成 ESP 栈顶下移, 以及 `src` 源操作数写入 `[ESP]`.

2. pop

```

1  // data-mov.c
2  make_EHelper(pop) {
3      /* dummy 中的 POP r32 使用 opcode 58+r. 先通过 RTL 弹出栈顶值,
4       * 再用通用 operand_write() 写回已经译码出的目标寄存器。
5       */
6      rtl_pop(&t2);
7      operand_write(id_dest, &t2);
8
9      print_asm_template1(pop);
10 }

```

通过 `rtl_pop(&t2)` 弹栈, 再 `operand_write()` 写回寄存器。

3. sub

```

1  // arith.c
2  make_EHelper(sub) {
3      /* DEST <- DEST - SRC; 再根据操作数宽度下的结果更新
4       * CF、OF、ZF、SF。*/
5      rtl_sub(&t2, &id_dest->val, &id_src->val);
6      operand_write(id_dest, &t2);
7
8      rtl_update_ZFSF(&t2, id_dest->width);
9
10     /* CF 记录无符号减法是否发生借位。先对两个操作数按宽度截断,
11      * 这样字节/字形式只比较体系结构上可见的低位。
12      */
13     rtl_li(&t0, rtl_mask(id_dest->width));
14     rtl_and(&t1, &id_dest->val, &t0);
15     rtl_and(&t3, &id_src->val, &t0);
16     rtl_sltu(&t0, &t1, &t3);
17     rtl_set_CF(&t0);
18
19     /* OF 记录有符号减法是否溢出:
20      * sign(DEST) != sign(SRC) 且 sign(DEST) != sign(RESULT)。
21      */
22     rtl_xor(&t0, &id_dest->val, &id_src->val);
23     rtl_xor(&t1, &id_dest->val, &t2);
24     rtl_and(&t0, &t0, &t1);
25     rtl_msb(&t0, &t0, id_dest->width);
26     rtl_set_OF(&t0);

```

```

26
27     print_asm_template2(sub);
28 }

```

DEST = DEST - SRC 计算写回，并更新 CF/OF/ZF/SF 标志位。

4. xor

```

1 // logic.c
2 make_EHelper(xor) {
3     /* dummy 中的 XOR Ev,Gv 使用 opcode 31 /r。i386 语义是
4      * DEST <- DEST ^ SRC，清零 CF/OF，并按结果宽度更新 ZF/SF。
5      */
6     rtl_xor(&t2, &id_dest->val, &id_src->val);
7     operand_write(id_dest, &t2);
8
9     rtl_li(&t0, 0);
10    rtl_set_CF(&t0);
11    rtl_set_OF(&t0);
12    rtl_update_ZFSF(&t2, id_dest->width);
13
14    print_asm_template2(xor);
15 }

```

DEST = DEST ^ SRC 计算写回，清除 CF/OF，更新 ZF/SF。

5. call

```

1 // control.c
2 make_EHelper(call) {
3     // the target address is calculated at the decode stage
4     /* CALL rel32
5      * 先压入下一条指令地址，再跳转到译码阶段算出的相对目标地址。
6      * 走到这里时 decode_J 已经取走 32 位位移，因此 *eip
7      * 已经是顺序执行的
8      * 下一条指令地址。
9      */
10    rtl_push(eip);
11    decoding.is_jump = 1;
12
13    print_asm("call %x", decoding.jump_eip);
14 }

```

通过 rtl_push(eip) 压入下一条指令地址，然后 decoding.is_jump = 1 跳到 decode_J 算好的目标地址。

6. ret

```

1 // control.c
2 make_EHelper(ret) {
3     /* 近返回 RET 从栈顶弹出返回地址作为新的 EIP。公共的 update_eip()

```

```

4      * 会在本 helper 返回后根据 is_jump 把 decoding.jump_eip 写入
      cpu.eip。
5      */
6      rtl_pop(&decoding.jump_eip);
7      decoding.is_jump = 1;
8
9      print_asm("ret");
10 }

```

通过 `rtl_pop(&decoding.jump_eip)` 弹出返回地址，再通过统一跳转路径更新 EIP。

`make_EHelper()` 函数是通过 `exec.h` 中的宏来定义的：

```
1 #define make_EHelper(name) void concat(exec_, name) (vaddr_t *eip)
```

所以以上所有这些 `make_EHelper(xxx)` 最终都会被展开为 `void exec_xxx(vaddr_t *eip)` (`xxx` 替换成真正的指令名)，这是某一条指令的**执行函数**。

这些函数什么时候被调用？再看 `exec.c` 中定义的 `opcode` 表项：

```

1 typedef struct {
2     DHelper decode;
3     EHelper execute;
4     int width;
5 } opcode_entry;

```

其中 `execute` 就是 `exec_xxx()` 的函数指针，对应“EHelper”的指针类型。当 `opcode` 被解析，在表中找到对应的表项后，在解析工具函数 `idex()` 中，`opcode_entry` 所含的函数指针就生效了，能够指引相应的执行函数被调用。这当中 `decode` 也是函数指针，指向解码函数，调用原理和执行函数类似。

5. 测试

以上代码实现完毕后，测试 `dummy` 的运行效果：

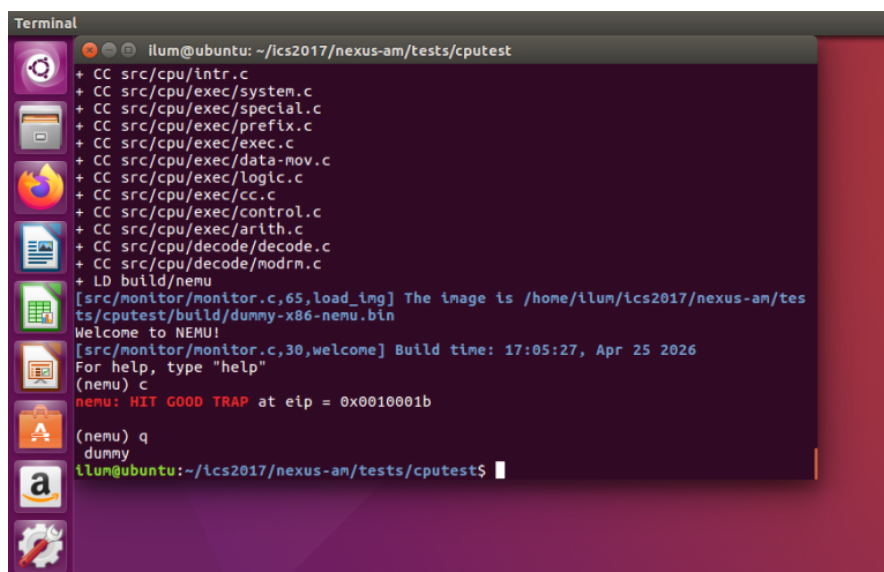


图 7: dummy 运行结果

在测试目录的 build 文件夹下会生成执行过程记录的汇编码：

```
[src/monitor/monitor.c,65,load_img] The image is /home/ilum/ics2017/nexus-
am/tests/cputest/build/dummy-x86-nemu.bin
[src/monitor/monitor.c,30,welcome] Build time: 17:05:27, Apr 25 2026
100000: bd 00 00 00 00          movl $0x0,%ebp
100005: bc 00 7c 00 00          movl $0x7c00,%esp
10000a: e8 01 00 00 00          call 100010
100010: 55                      pushl %ebp
100011: 89 e5                  movl %esp,%ebp
100013: 83 ec 08              subl $0x8,%esp
100016: e8 05 00 00 00          call 100020
100020: 55                      pushl %ebp
100021: 89 e5                  movl %esp,%ebp
100023: 31 c0                  xorl %eax,%eax
100025: 5d                      popl %ebp
100026: c3                      ret
10001b: d6                      nemu trap (eax = 0)
```

可以看到 dummy 执行了所有 C 程序逻辑并且最终到达了 nemu 设置的 GOOD TRAP.

(二) CHECKPOINT: PA2 stage1

至此 PA2 stage1 完成。

手动 git 本地记录如下（最顶上一次最新的是手动 git 记录）：

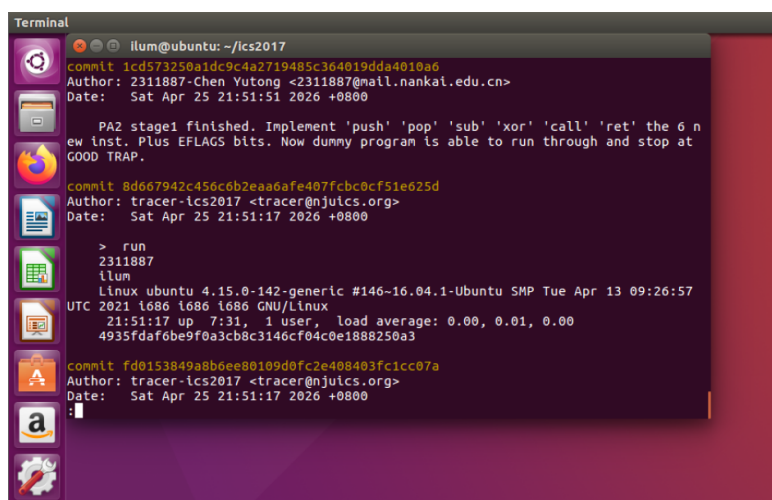


图 8: 手动 git 记录

(三) TODO: 实现更多的指令

其他指令的实现流程和上面 6 条指令的基本流程是一致的。

全局来看, 依旧是在 `exec.c` 将指令注册到 `opcode` 表中, 然后再在 `all-instr.h` 补全所有 helper 的声明。

```

1 // 只展示部分作为例子
2
3 /* group 1: 立即数/ModR/M 共享的算术逻辑指令组 */
4 make_group(gp1,
5     EX(add), EX(or), EX(adc), EX(sbb),
6     EX(and), EX(sub), EX(xor), EX(cmp))
7
8 /* group 2: 移位指令组 */
9 make_group(gp2,
10    EMPTY, EMPTY, EMPTY, EMPTY,
11    EX(shl), EX(shr), EMPTY, EX(sar))
12
13 /* group 3: 单操作数算术逻辑指令组 */
14 make_group(gp3,
15    IDEX(test_I, test), EMPTY, EX(not), EX(neg),
16    EX(mul), EX(imul1), EX(div), EX(idiv))
17
18 /* ... */
19
20 opcode_entry opcode_table[512] = {
21    /* 0x00 */ IDEXW(G2E, add, 1), IDEX(G2E, add), IDEXW(E2G, add,
22    1), IDEX(E2G, add),
23    /* 0x04 */ IDEXW(I2a, add, 1), IDEX(I2a, add), EMPTY, EMPTY,
24    /* 0x08 */ IDEXW(G2E, or, 1), IDEX(G2E, or), IDEXW(E2G, or, 1),
25    IDEX(E2G, or),
26    /* 0x0c */ IDEXW(I2a, or, 1), IDEX(I2a, or), EMPTY, EX(2byte_esc),
27    /* 0x10 */ IDEXW(G2E, adc, 1), IDEX(G2E, adc), IDEXW(E2G, adc,
28    1), IDEX(E2G, adc),
29    /* 0x14 */ IDEXW(I2a, adc, 1), IDEX(I2a, adc), EMPTY, EMPTY,
30    /* 0x18 */ IDEXW(G2E, sbb, 1), IDEX(G2E, sbb), IDEXW(E2G, sbb,
31    1), IDEX(E2G, sbb),
32    /* 太长不一一展示... 完整的表见源代码 */

```

注册 `opcode` 表后, 根据指令类型在不同的.c 文件下实现 helper 函数。

helper 函数新增/修改统计有:

1. data-mov.c

实现了 `leave`、`cldt(cwd/cdq)`、`cwt1(cbtw/cwde)`, 并修正了 `cldt` 在 16 位前缀下的反汇编打印。

```

1 make_EHelper(leave) {
2     /* leave 等价于:
3     *     mov esp, ebp

```

```

4      *   pop ebp
5      * 这样函数尾声可以一次恢复当前栈帧。 */
6      rtl_mv(&cpu.esp, &cpu.ebp);
7      rtl_pop(&cpu.ebp);
8
9      print_asm("leave");
10 }
11
12 make_EHelper(cltd) {
13     if (decoding.is_operand_size_16) {
14         /* cwd: 用 AX 的符号位填满 DX。 */
15         rtl_lr_w(&t0, R_AX);
16         rtl_sext(&t1, &t0, 2);
17         rtl_sari(&t1, &t1, 16);
18         rtl_sr_w(R_DX, &t1);
19     }
20     else {
21         /* cdq: 用 EAX 的符号位填满 EDX。 */
22         rtl_lr_l(&t0, R_EAX);
23         rtl_sext(&t1, &t0, 4);
24         rtl_sari(&t1, &t1, 31);
25         rtl_sr_l(R_EDX, &t1);
26     }
27
28     print_asm(decoding.is_operand_size_16 ? "cwd" : "cltd");
29 }
30
31 make_EHelper(cwtl) {
32     if (decoding.is_operand_size_16) {
33         /* cbtw: 把 AL 符号扩展到 AX。 */
34         rtl_lr_b(&t0, R_AL);
35         rtl_sext(&t1, &t0, 1);
36         rtl_sr_w(R_AX, &t1);
37     }
38     else {
39         /* cwtl/cwde: 把 AX 符号扩展到 EAX。 */
40         rtl_lr_w(&t0, R_AX);
41         rtl_sext(&t1, &t0, 2);
42         rtl_sr_l(R_EAX, &t1);
43     }
44
45     print_asm(decoding.is_operand_size_16 ? "cbtw" : "cwtl");
46 }

```

2. arith.c

实现/补全了 add/inc/sub/dec/cmp/neg, 并把 CF/OF/ZF/SF 位的更新单独写成函数接口。

```
1 static inline void update_CF_add(const rtlreg_t *dest, const
    rtlreg_t *result, int width) {
2     /* 加法的 CF
        等价于“按当前操作数宽度截断后，结果是否小于原目的操作数”。 */
3     rtlreg_t mask, dest_masked, result_masked;
4     rtl_li(&mask, rtl_mask(width));
5     rtl_and(&dest_masked, dest, &mask);
6     rtl_and(&result_masked, result, &mask);
7     rtl_sltu(&t0, &result_masked, &dest_masked);
8     rtl_set_CF(&t0);
9 }
10
11 static inline void update_CF_adc(const rtlreg_t *dest, const
    rtlreg_t *src, const rtlreg_t *carry, int width) {
12     /* ADC 的 CF 需要同时考虑两次无符号进位：
        * 第一次是 DEST + SRC，第二次是在部分和上继续加旧 CF。 */
13     /* 代码较长此处略 */
14     /* ... */
15 }
16
17
18 static inline void update_CF_sub(...) {
19     /* 减法的 CF 表示无符号借位，因此只需比较截断后的 DEST 是否小于
        SRC。 */
20     /* 具体代码略 */
21     /* ... */
22 }
23
24 static inline void update_CF_sbb(...) {
25     /* SBB 的 CF 同样拆成两步判断：
        * 先看 DEST - SRC 是否借位，再看减去旧 CF 时是否继续借位。 */
26     /* 代码较长此处略 */
27     /* ... */
28 }
29
30
31 static inline void update_OF_add(...) {
32     /* 有符号加法溢出条件：
        * sign(DEST) == sign(SRC) 且 sign(DEST) != sign(RESULT)。 */
33     /* 具体代码略 */
34     /* ... */
35 }
36
37
38 static inline void update_OF_sub(...) {
39     /* 有符号减法溢出条件：
        * sign(DEST) != sign(SRC) 且 sign(DEST) != sign(RESULT)。 */
40     /* 具体代码略 */
41     /* ... */
42 }
43
44 }
```

```
45 make_EHelper(add) {
46     /* ADD 会写回结果，并统一通过辅助函数更新 ZF/SF/CF/OF。 */
47     rtl_add(&t2, &id_dest->val, &id_src->val);
48     operand_write(id_dest, &t2);
49
50     rtl_update_ZFSF(&t2, id_dest->width);
51     update_CF_add(&id_dest->val, &t2, id_dest->width);
52     update_OF_add(&id_dest->val, &id_src->val, &t2, id_dest->width);
53
54     print_asm_template2(add);
55 }
56
57 make_EHelper(sub) {
58     /* SUB 会写回结果；具体的借位与溢出判断复用上面的通用 helper。 */
59     rtl_sub(&t2, &id_dest->val, &id_src->val);
60     operand_write(id_dest, &t2);
61
62     rtl_update_ZFSF(&t2, id_dest->width);
63     update_CF_sub(&id_dest->val, &id_src->val, id_dest->width);
64     update_OF_sub(&id_dest->val, &id_src->val, &t2, id_dest->width);
65
66     print_asm_template2(sub);
67 }
68
69 make_EHelper(cmp) {
70     /* CMP 和 SUB 的标志位行为一致，只是它不写回结果。 */
71     /* 具体代码略 */
72     /* ... */
73 }
74
75 make_EHelper(inc) {
76     rtlreg_t one;
77     rtl_li(&one, 1);
78
79     /* INC 只等价于加 1，但按 i386 语义不会修改 CF。 */
80     rtl_add(&t2, &id_dest->val, &one);
81     operand_write(id_dest, &t2);
82
83     rtl_update_ZFSF(&t2, id_dest->width);
84     update_OF_add(&id_dest->val, &one, &t2, id_dest->width);
85
86     print_asm_template1(inc);
87 }
88
89 make_EHelper(dec) {
90     /* DEC 只等价于减 1，同样不会修改 CF。 */
91     /* 类似 inc 写法，具体代码略 */
92     /* ... */
```

```
93 }
94
95 make_EHelper(neg) {
96     /* NEG 等价于 0 - DEST。
97     * 当原操作数非 0 时，CF 需要被置 1。 */
98     rtl_sub(&t2, &tzero, &id_dest->val);
99     operand_write(id_dest, &t2);
100
101     rtl_update_ZFSF(&t2, id_dest->width);
102     rtl_neq0(&t0, &id_dest->val);
103     rtl_set_CF(&t0);
104     update_OF_sub(&tzero, &id_dest->val, &t2, id_dest->width);
105
106     print_asm_template1(neg);
107 }
```

3. logic.c

实现了 test/and/or/xor/not/shl/shr/sar。

```
1 static inline void clear_logic_flags(void) {
2     /* 逻辑类指令执行后会把 CF 和 OF 清 0。 */
3     rtl_li(&t0, 0);
4     rtl_set_CF(&t0);
5     rtl_set_OF(&t0);
6 }
7
8 make_EHelper(test) {
9     /* TEST 只做按位与并更新标志位，不写回结果。 */
10    rtl_and(&t2, &id_dest->val, &id_src->val);
11    clear_logic_flags();
12    rtl_update_ZFSF(&t2, id_dest->width);
13
14    print_asm_template2(test);
15 }
16
17 make_EHelper(and) {
18     /* AND 会写回按位与结果，并按 i386 语义清零 CF/OF。 */
19    rtl_and(&t2, &id_dest->val, &id_src->val);
20    operand_write(id_dest, &t2);
21
22    clear_logic_flags();
23    rtl_update_ZFSF(&t2, id_dest->width);
24
25    print_asm_template2(and);
26 }
27
28 make_EHelper(or) {
29     /* OR 会写回按位或结果，并按 i386 语义清零 CF/OF。 */
```

```

30  /* 类似 AND, 具体代码略 */
31  /* ... */
32  }
33
34  make_EHelper(sar) {
35      /* SAR 是算术右移, 高位补符号位; 当前实验只需要更新 ZF/SF。 */
36      rtl_sar(&t2, &id_dest->val, &id_src->val);
37      operand_write(id_dest, &t2);
38      rtl_update_ZFSF(&t2, id_dest->width);
39
40      print_asm_template2(sar);
41  }
42
43  make_EHelper(shl) {
44      /* SHL/SAL 是逻辑左移; 当前实验只需要更新 ZF/SF。 */
45      /* 类似 SAR, 具体代码略 */
46      /* ... */
47  }
48
49  make_EHelper(shr) {
50      /* SHR 是逻辑右移, 高位补 0; 当前实验只需要更新 ZF/SF。 */
51      /* 类似, 具体代码略 */
52      /* ... */
53  }
54
55  make_EHelper(not) {
56      /* NOT 只做按位取反, 不影响任何标志位。 */
57      rtl_mv(&t2, &id_dest->val);
58      rtl_not(&t2);
59      operand_write(id_dest, &t2);
60
61      print_asm_template1(not);
62  }

```

4. control.c

实现了 call_rm。

```

1  make_EHelper(call_rm) {
2      /* 间接 call 会先把顺序执行的下一条指令地址压栈,
3       * 再跳到译码阶段算出的目标操作数。 */
4      rtl_push(eip);
5      decoding.jump_eip = id_dest->val;
6      decoding.is_jump = 1;
7
8      print_asm("call %s", id_dest->str);
9  }

```

除此之外, 还对于 decode.c 中 push imm8 相关进行了修正:

```

1 make_DHelper(push_SI) {
2     int width = id_dest->width;
3
4     /* PUSH imm8 的编码只有 1
       字节立即数，但真正压栈前需要先按操作数宽度做符号扩展。 */
5     id_dest->width = 1;
6     decode_op_SI(eip, id_dest, true);
7     id_dest->width = width;
8
9     if (width == 2) {
10         id_dest->val &= 0xffff;
11     }
12 }

```

修正了 push imm8 的译码：先按 1 字节取立即数，再按操作数宽度做符号扩展。这是手册中强调的一个易错点。（“If the source operand is an immediate and its size is less than the operand size, a sign-extended value is pushed on the stack.”——如果源操作数是立即数，且大小小于操作数大小，那么在压栈之前，应先对该值进行**符号扩展**。）

再者，对于 jcc 和 setcc 两条条件码指令的条件成立判断逻辑，原代码框架将它们用 cc.c 集中封装了起来，这里需要将各条件分支内的具体代码补全：

```

1 // dest <- ( cc is satisfied ? 1 : 0)
2 switch (subcode & 0xe) {
3     case CC_0:
4         rtl_get_OF(dest);
5         break;
6     case CC_B:
7         rtl_get_CF(dest);
8         break;
9     case CC_E:
10        rtl_get_ZF(dest);
11        break;
12     case CC_BE:
13        rtl_get_CF(dest);
14        rtl_get_ZF(&t0);
15        rtl_or(dest, dest, &t0);
16        break;
17     case CC_S:
18        rtl_get_SF(dest);
19        break;
20     case CC_L:
21        rtl_get_SF(dest);
22        rtl_get_OF(&t0);
23        rtl_xor(dest, dest, &t0);
24        break;
25     case CC_LE:
26        rtl_get_ZF(dest);

```



```

27     rtl_get_SF(&t0);
28     rtl_get_OF(&t1);
29     rtl_xor(&t0, &t0, &t1);
30     rtl_or(dest, dest, &t0);
31     break;
32 default: panic("should not reach here");
33 case CC_P: panic("n86 does not have PF");
34 }

```

最终，以上代码完整实现了以下所有指令并可在 NEMU 测试中使用：

- 数据传送：mov, push, pop, leave, movsx, movzx, lea, nop
- 算术：add, inc, sub, dec, cmp, neg, adc, sbb, mul, imul(1/2/3), div, idiv
- 逻辑/位移：not, and, or, xor, sar, shr, shl/sal, setcc, test
- 控制转移：jmp, jcc, call, ret, call_rm, jmp_rm

按照手册要求在 Makefile.check 中更改：

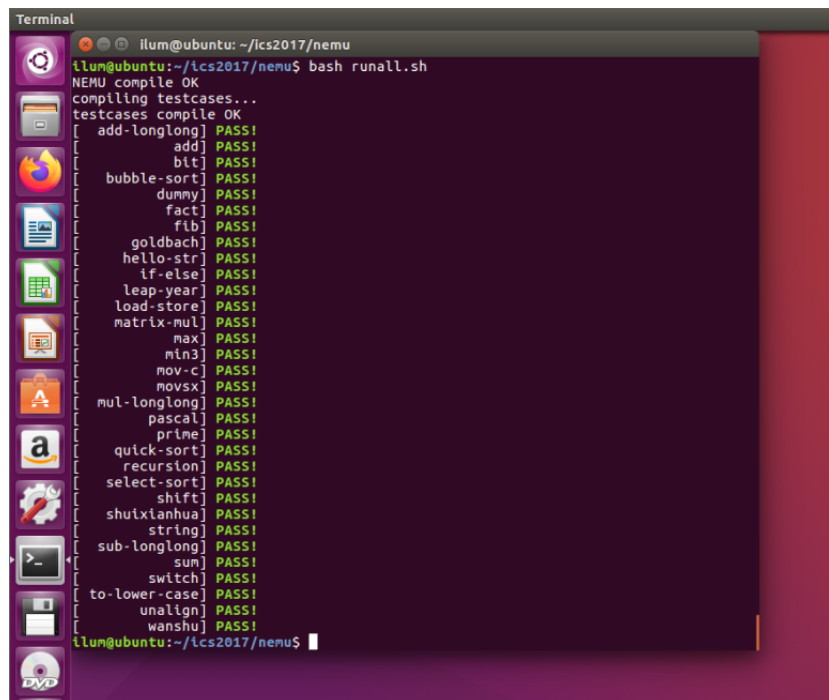
```

-ARCH ?= native
+ARCH ?= x86-nemu

```

——让 AM 项目上的程序默认编译到 x86-nemu 的 AM 中，而后通过 `make ALL=xxx run` 运行 nexus-am/tests/cputest/tests 目录下的所有测试样例。

按照后续指导手册的指示，也可以在 nemu/ 目录下通过 `bash runall.sh` 一键测试所有上面提到目录下的 test，当前代码实现状态下跑出的测试结果如图所示：



```

Terminal
ilum@ubuntu: ~/ics2017/nemu
ilum@ubuntu:~/ics2017/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
[ string] PASS!
[ sub-longlong] PASS!
[ sum] PASS!
[ switch] PASS!
[ to-lower-case] PASS!
[ unalign] PASS!
[ wanshu] PASS!
ilum@ubuntu:~/ics2017/nemu$

```

图 9: 运行 runall.sh 测试脚本结果

测试样例全部通过。新增指令实现正确。

(四) TODO: 实现 differential testing

在 `difftest_step()` 函数中补全寄存器状态比对的代码逻辑, 对于 8 个 GDB 通用寄存器直接按序对比即可; 对于 EIP 单独取出比较 (它不在 NEMU 定义的 `cpu.gpr[]` 内):

```

1 void difftest_step(uint32_t eip) {
2     /* initiate ... */
3
4     // PA 2: QEMU 的 8 个 GDB 通用寄存器顺序构建和 cpu.gpr[] 一致
5     int i;
6     for (i = R_EAX; i <= R_EDI; i++) {
7         if (reg_l(i) != r.array[i]) {
8             printf("DIFF at eip = 0x%08x: %s, NEMU = 0x%08x, QEMU =
9                 0x%08x\n",
10                 eip, reg_name(i, 4), reg_l(i), r.array[i]);
11             diff = true;
12         }
13     }
14
15     // EIP 不在 cpu.gpr[] 内, 需要单独比较
16     if (cpu.eip != r.eip) {
17         printf("DIFF at eip = 0x%08x: eip, NEMU = 0x%08x, QEMU =
18             0x%08x\n",
19             eip, cpu.eip, r.eip);
20         diff = true;
21     }
22
23     if (diff) {
24         nemu_state = NEMU_END;
25     }
26 }

```

打开 `common.h` 中的 `diff test` 宏, 运行效果:

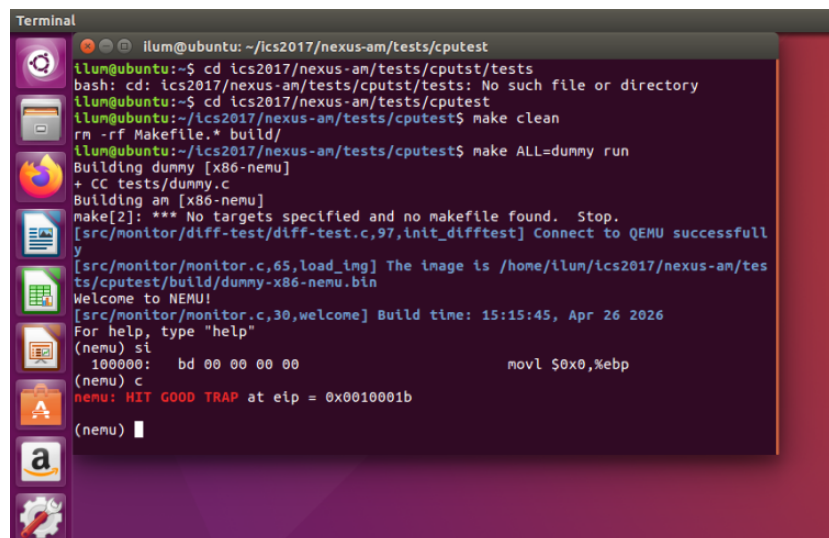


图 10: diff test 开启效果

启动时会显示 “Connect to QEMU successfully”，表示已经成功启动了 QEMU. 运行到 GOOD TRAP 过程中一直没有报错形如 “DIFF at eip = ... :” 说明指令全部执行、每条指令都与 QEMU 进行了 differential testing 并验证为寄存器状态相同、也就是 NEMU 实现正确。

(五) CHECKPOINT: PA2 stage2

至此 PA2 stage2 完成。

手动 git 本地记录如下（最顶上一次最新的是手动 git 记录）：

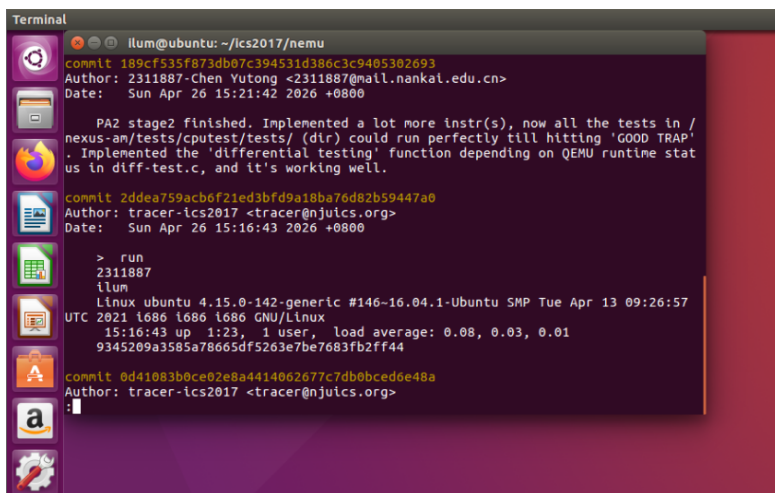


图 11: 手动 git 记录

(六) TODO: 实现 in/out 指令并运行 Hello World

直接先开启 define HAS_SERIAL 然后 make run, 在模拟有串口的条件下, 从汇编代码记录就可以知道 in 和 out 指令的 opcode 分别是 0xec 和 0xee. 而后再和之前类似的流程去实现这两个指令。

在 opcode 表注册的代码不再展示。in 和 out 指令的 helper 函数实现都在 system.c, 原代码框架已经准备好了接口, 补全如下:

```

1 make_EHelper(in) {
2     /* PA 2: IN 根据操作数宽度, 从译码得到的 I/O 端口读取数据到
3        AL/AX/EAX. */
4     t2 = pio_read(id_src->val, id_dest->width);
5     operand_write(id_dest, &t2);
6
7     print_asm_template2(in);
8
9     #ifdef DIFF_TEST
10    diff_test_skip_qemu();
11 #endif
12 }
13 make_EHelper(out) {

```

```

14  /* PA 2: OUT 将 AL/AX/EAX 写入译码得到的 I/O 端口，端口号来自
    imm8 或 DX。 */
15  pio_write(id_dest->val, id_src->width, id_src->val);
16
17  print_asm("out%c %s,%s", suffix_char(id_src->width), id_src->str,
    id_dest->str);
18
19  #ifdef DIFF_TEST
20      diff_test_skip_qemu();
21  #endif
22  }

```

这当中，用到了 `pio_read()` 和 `pio_write()` 两个定义在 `port-io.c` 的固有 CPU 接口，能够从 I/O 端口读/写数据。输入时的读取还需要将结果写回操作数（这里是寄存器），所以 `in` 指令还用到了 `operand_write()` 函数。

实现 `in/out` 指令后运行 Hello World 程序测试：

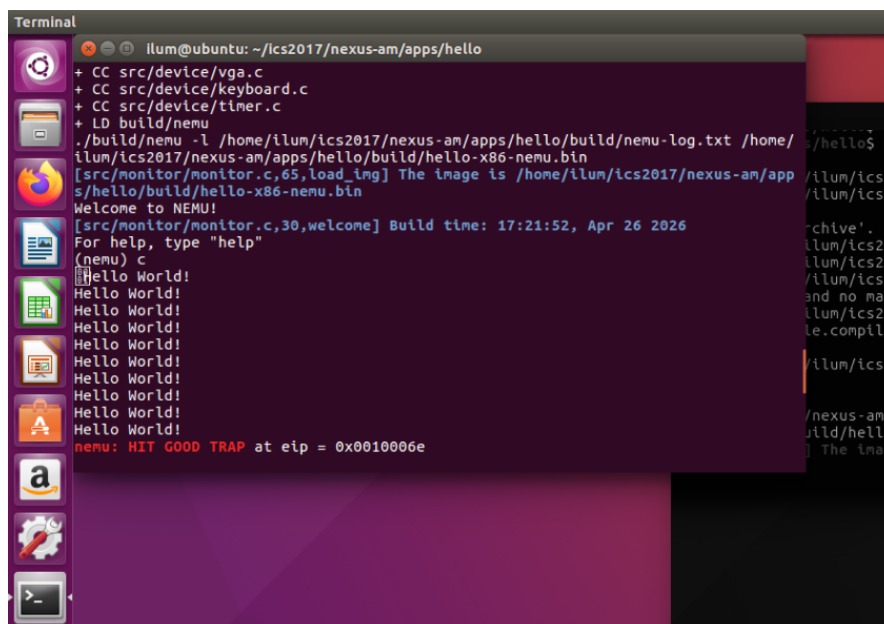


图 12: hello world 运行结果

打开串口后运行 NEMU 时会弹出一个**新窗口**，用于显示 VGA 输出。但 `hello` 这个 AM 程序只通过串口 `_putc()` 输出，不会往 VGA 显存打印东西，所以图形窗口不显示任何内容，这是合理的。

但除此之外，发现 console 输出的开头有一个乱码，这是不正常的。

排查原因：

`trm.c` 中 `outb(SERIAL_PORT + 0, 0x01);`，这一句在真实 16550 串口里是用来设置波特率 divisor low byte 的，因为前面有 `outb(SERIAL_PORT + 3, 0x80);` 即设置了 DLAB 位有效（串口的 LCR/线路控制寄存器中的一位，当 DLAB=1 时，`0x3f8/0x3f9` 用来设置波特率除数），此时 `0x01` 虽然被写入到 `SERIAL_PORT + 0 (= 0x3f8)` 却不应该被当做需要输出到 console 的内容。但当前代码 `serial.c` 中只要看到写 `SERIAL_PORT + 0`，就会直接 `putc()` 打印到宿主终端，所以把 `0x01` 当字符打出来了，显示出来是个不可解读的字符。

可以在 serial.c 中添加极少量的代码来修复：

```

1 #define LCR_OFFSET 3
2 /* ... */
3 void serial_io_handler(ioaddr_t addr, int len, bool is_write) {
4     /* ... */
5     -- if (addr == SERIAL_PORT + CH_OFFSET) {
6     ++ if (addr == SERIAL_PORT + CH_OFFSET &&
        (serial_port_base[LCR_OFFSET] & 0x80) == 0) {

```

其含义是：只有 LCR 的 DLAB 位为 0 时，写 0x3f8 才当成串口发送字符。这样一来，初始化阶段 outb(0x3f8, 0x01) 时 DLAB=1，就不会再把 0x01 打到终端。

修改后重新 make run，效果：

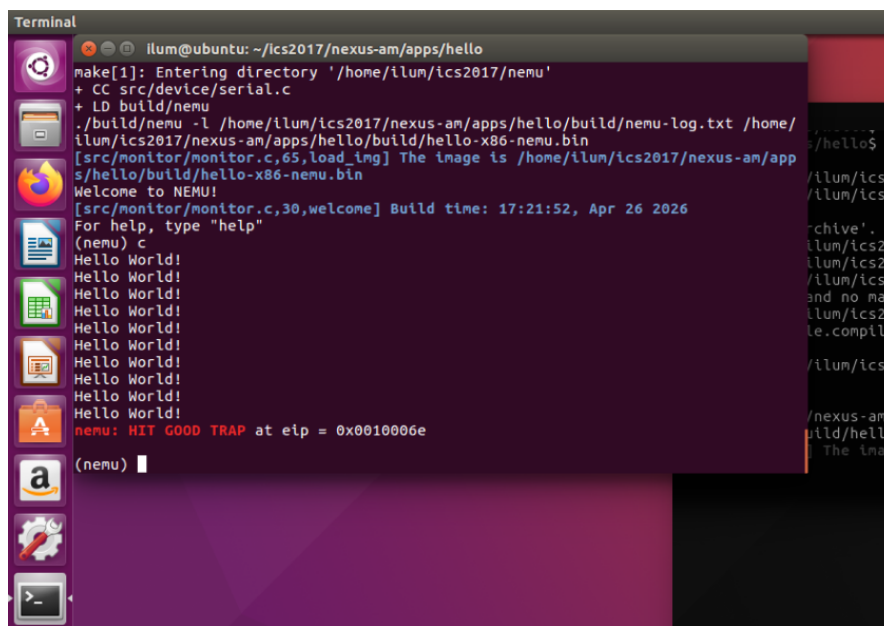


图 13: 修复后 hello world 运行结果

首部不再有乱码。

(七) TODO: 实现 ioe (1) - __uptime()

在 ioe.c 中补全 __uptime() 如下所示：

```

1 unsigned long __uptime() {
2     /* PA 2: RTC_PORT 返回 AM 程序启动后的运行时间 (ms) */
3     return inl(RTC_PORT) - boot_time;
4 }

```

运行 timetest 程序效果：

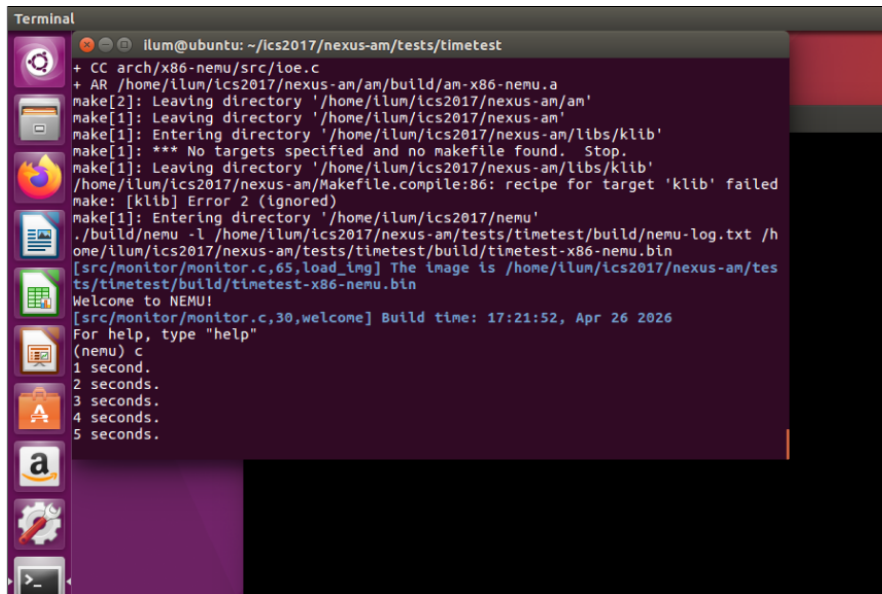


图 14: uptime 功能验证

每隔一秒输出一条计时信息。

(八) TASK: 用时钟测试计算机性能

指导手册要求用刚刚补全的时钟功能来测试 NEMU 的性能。为了多一些参考对比，这里采用 x86-nemu 和 native 本机环境均测试的方案，直接编译（可使用 ARCH=xxx 切换 nemu 和 native）运行相应的测试程序即可，以下是测试结果数据。

1. Dhrystone

x86-nemu: 124 分 (0.1%)

native: 39625 $\approx$ 40000 分 (40%)

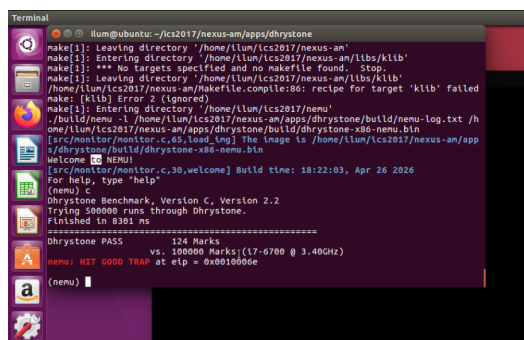


图 15: x86-nemu

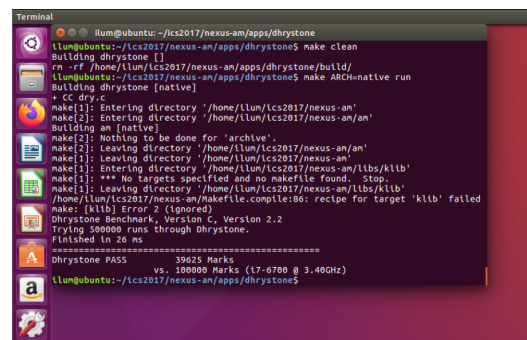


图 16: native

2. Coremark

x86-nemu: 435 分 (0.4%)

native: 63837 $\approx$ 64000 分 (64%)

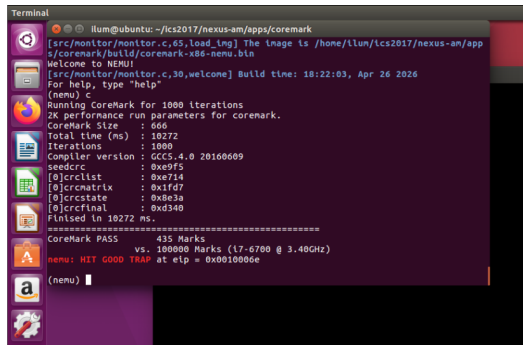


图 17: x86-nemu

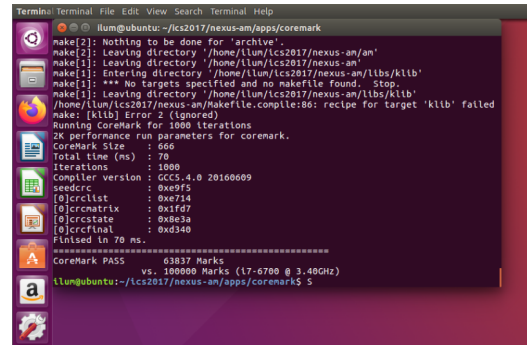


图 18: native

3. Microbench

x86-nemu: 377 $\approx$ 400 分 (0.4%)

native: 64031 分 (64%)

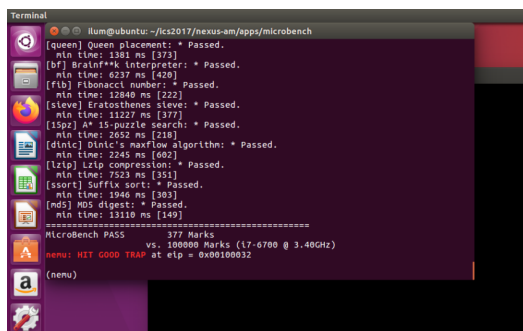


图 19: x86-nemu

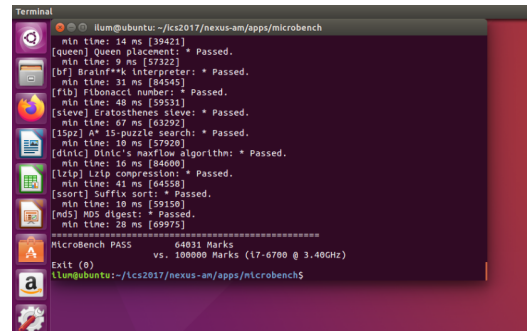


图 20: native

注: "native" 是指操作系统默认的运行时环境, 在这里就是 Ubuntu 虚拟机。

根据上面的数据, 比较 x86-nemu 和 native (相较于参照机) 的性能, 分析两者差异及原因:

1. ARCH=native 时, 程序被编译成宿主机能直接执行的机器码。加法、访存、跳转这些指令都由真实 CPU 直接完成, 还能得到流水线、缓存、分支预测、乱序执行等硬件优化的辅助。参照机的性能相比于当前实验的宿主机可能较高, 并且还要考虑到 Ubuntu 虚拟机这一层缓冲, 包括可能未启用完整的编译参数与选项等——最终宿主机表现出的性能是参照机的 40~60%, 这也在合理范围内。
2. ARCH=x86-nemu 时, 是通过 NEMU 代码的软件模拟。在 NEMU 里, 客户程序的一条 x86 指令不是被 CPU 直接执行, 而是经历类似过程“取指 -> 译码 -> 执行 -> 更新模拟寄存器/内存/eflags -> 检查中断/设备”。这些步骤本身都是 C 程序代码。也就是说, 客户程序里一条简单指令, 经过 NEMU 这个中介, 可能需要宿主机执行几十条甚至上百条真实指令才能模拟出来, 所以性能只有 native 的千分之几 (0.1% 数量级) 也正常。

(九) TODO: 实现 ioe (2) - __read_key()

在 ioe.c 中补全 __read_key():

```
1 int __read_key() {
```

```

2  /*
3   * 先读状态端口：NEMU 的 i8042 用 bit 0 表示“有键盘事件”。
4   * 如果没有事件，按照 AM 的约定返回 _KEY_NONE。
5   */
6  if ((inb(I8042_STATUS_PORT) & I8042_STATUS_HASKEY_MASK) == 0) {
7      return _KEY_NONE;
8  }
9
10 // 再读数据端口：返回的就是 AM 键盘码，按下事件已经带有 0x8000
    标志。
11 return inl(I8042_DATA_PORT);
12 }

```

补全后运行 keytest 程序效果：

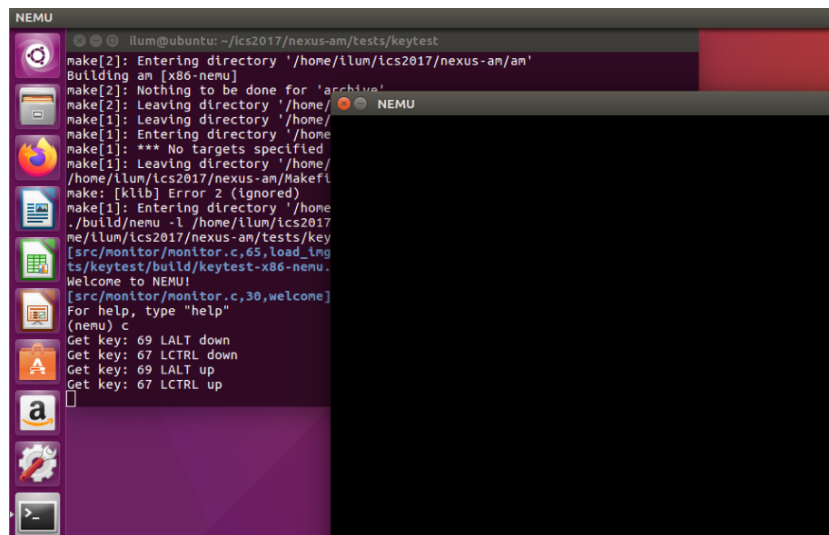


图 21: 键盘输入效果 1

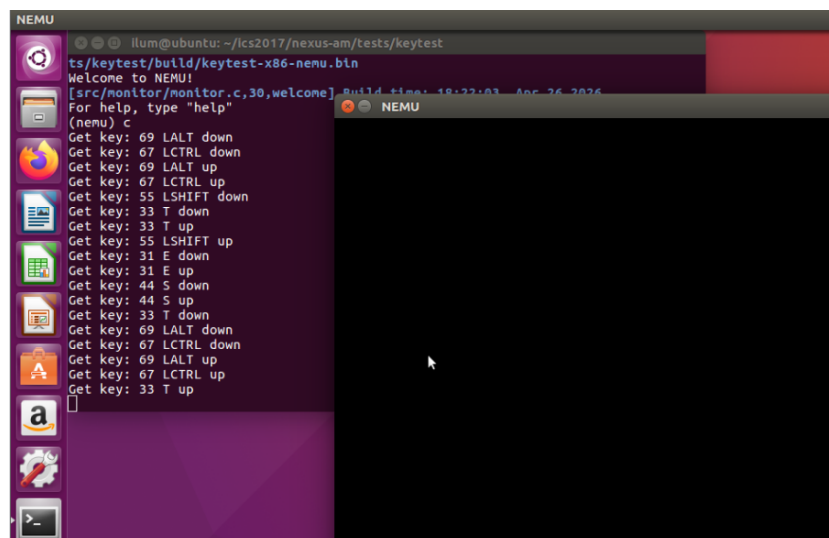


图 22: 键盘输入效果 2

随意按下并松开一些键，可以看到主窗口有打印出所有按键被按下和松开的信息。这是因为，对于键盘输入状态，NEMU 监听的是按下/松开的动作信号。按下信号到达，将对应键加入有效状态；松开信号到达，将对应键移除有效状态。

(十) TODO: 添加内存映射 I/O

memory.c 中, 在 `paddr_read()` 和 `paddr_write()` 中加入对内存映射 I/O 的判断。需要用到 `mmio.c` 中的 `is_mmio()` 来判断一个物理地址是否被映射到 I/O 空间、并调用 `mmio_read()` 或 `mmio_write()` 进行内存映射 I/O 的访问。如果不是内存映射 I/O 的访问, 则访问 `pmem`. 代码补全如下:

```
1 uint32_t paddr_read(paddr_t addr, int len) {
2     /*
3      * 物理地址可能落在设备通过 MMIO 映射出来的空间中。
4      * 如果 is_mmio() 找到了映射, 就交给设备的 MMIO 读接口处理;
5      * 否则才把它当作普通物理内存 pmem 来访问。
6      */
7     int map_NO = is_mmio(addr);
8     if (map_NO != -1) {
9         return mmio_read(addr, len, map_NO);
10    }
11
12    return pmem_rw(addr, uint32_t) & (~0u >> ((4 - len) << 3));
13 }
14
15 void paddr_write(paddr_t addr, int len, uint32_t data) {
16     /*
17      * MMIO 写入不能直接写
18      * pmem, 因为目标可能是显存等设备寄存器/缓冲区。
19      * 命中映射时调用 mmio_write(), 由对应设备的回调完成必要处理。
20      */
21     int map_NO = is_mmio(addr);
22     if (map_NO != -1) {
23         mmio_write(addr, len, data, map_NO);
24         return;
25     }
26
27     memcpy(guest_to_host(addr), &data, len);
28 }
```

补全后运行 videotest 程序效果:

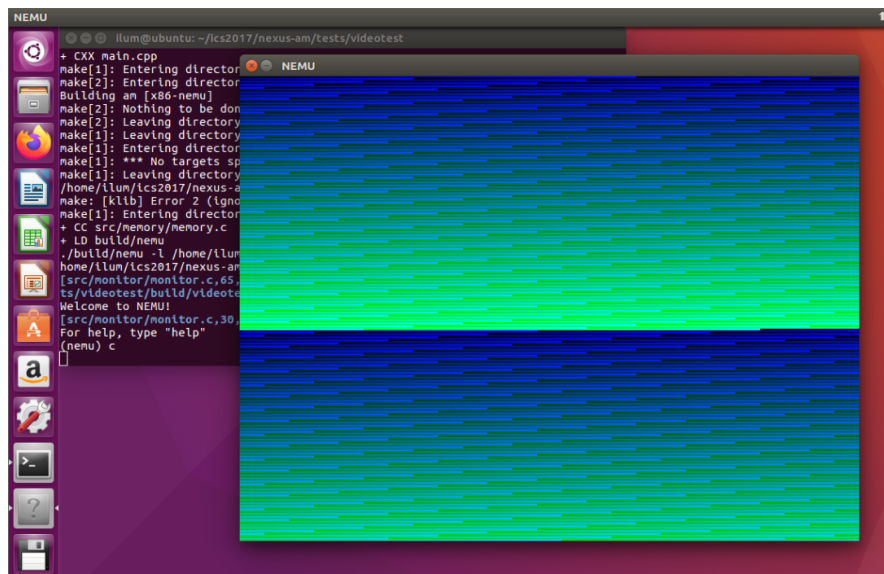


图 23: videotest 运行结果

和指导手册说的一样，有颜色信息的输出——但还没有任何意义，只是简单的平铺填充。

(十一) TODO: 实现 ioe (3) - `_draw_rect()`

`videotest` 每次会反复调用 `_draw_rect(color_buf, x*w, y*h, w, h)`，所以这里应该把传入的小矩形逐行拷贝到 `0x40000` 这块显存里，而不是像原来填整屏测试值。改进后的 `_draw_rect()`：

```

1 void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h)
2 {
3     /*
4      * fb 指向 NEMU 中映射到 0x40000 的显存。
5      * pixels 是一块 w*h 的连续像素数组，按行拷贝到屏幕上
6      * 以 (x, y) 为左上角的矩形区域即可。
7      */
8     int copy_w = w;
9     if (x + copy_w > _screen.width) {
10         copy_w = _screen.width - x;
11     }
12     for (int j = 0; j < h && y + j < _screen.height; j++) {
13         memcpy(&fb[(y + j) * _screen.width + x], pixels, copy_w *
14             sizeof(uint32_t));
15         pixels += w;
16     }
17 }

```

修改后再次运行 `videotest` 程序：

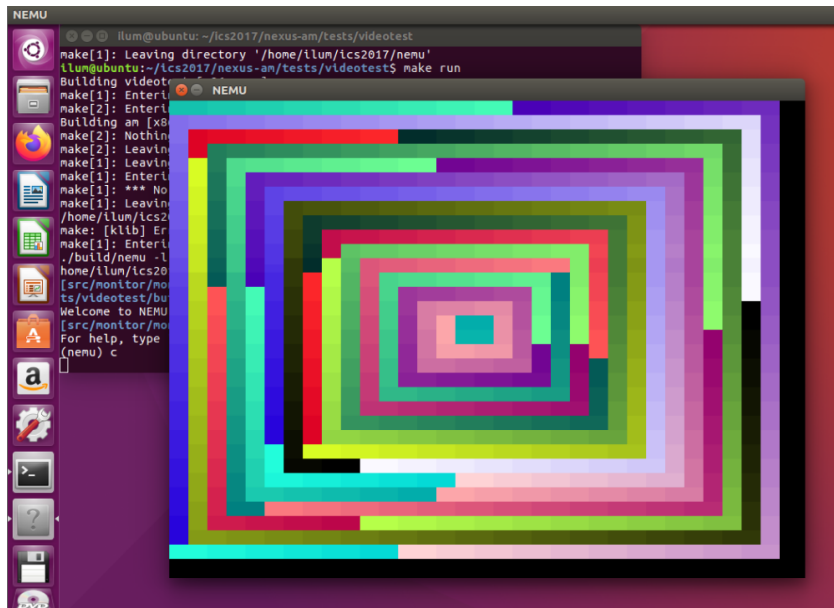


图 24: 改进后 videotest 运行结果 (时间点 1)

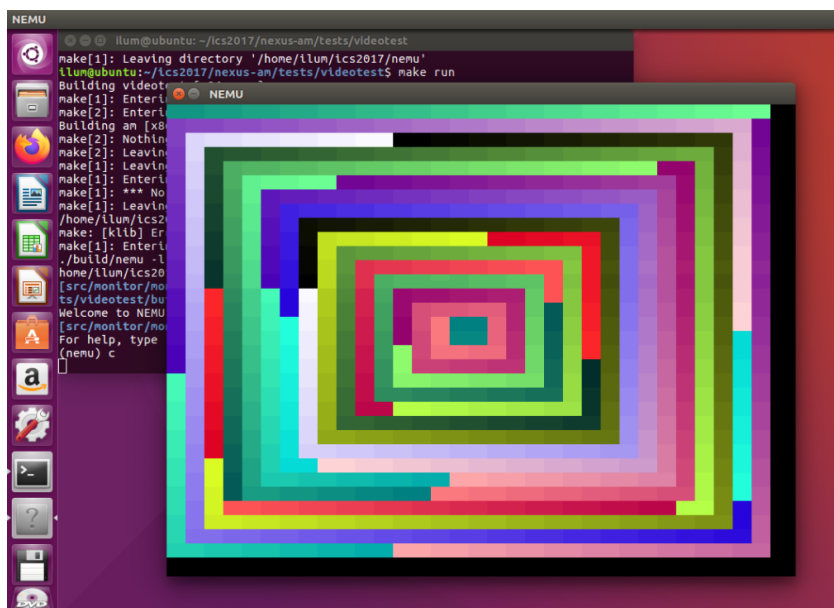


图 25: 改进后 videotest 运行结果 (时间点 2)

有很多种颜色有序填充，并且是动态变化的。

(十二) TODO: 运行打字小游戏

上面的代码实现全部完成后，运行 apps/typing 打字游戏：

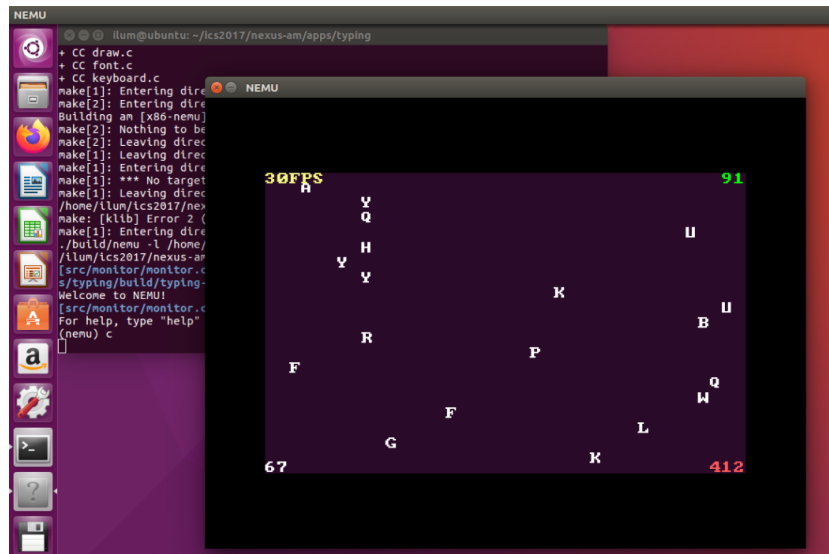


图 26: typing 运行界面

已经可以成功运行。

typing 小游戏的图形界面能够完整正确地显示、且键盘输入反馈有效。

还尝试运行了红白机模拟程序：



图 27: litenes 运行界面

能够成功运行。

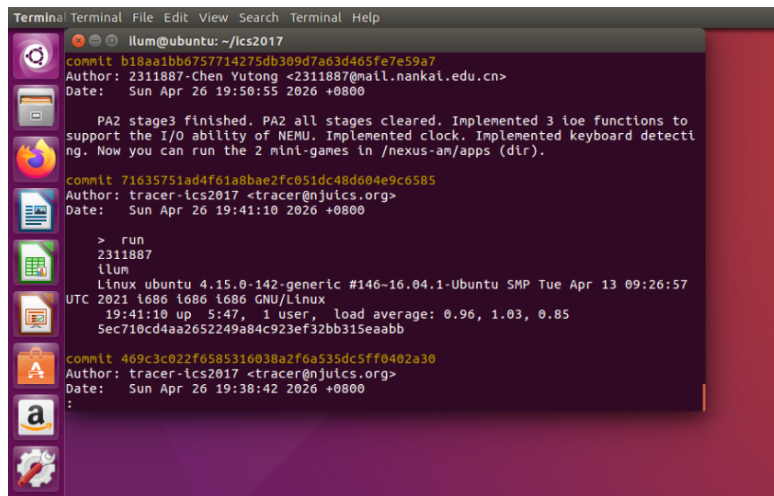
但是，可能因为 nemu 虚拟出来的 CPU 性能不佳，红白机运行时帧数非常低，键盘输入反馈延迟也非常大。

后续尝试换作 ARCH=native 进行编译链接并运行，就可以正常运行，反馈体验比较及时。这也印证了之前对于 x86-nemu 和 native 机器性能对比的结果。

(十三) CHECKPOINT: PA2 stage3

至此 PA2 stage3 完成。

手动 git 本地记录如下（最顶上一次最新的是手动 git 记录）：



```
Terminal Terminal File Edit View Search Terminal Help
ilum@ubuntu: ~/ics2017
commit b18aa1bb6757714275db309d7a63d465fe7e59a7
Author: 2311887-Chen Yutong <2311887@mail.nankai.edu.cn>
Date: Sun Apr 26 19:50:55 2026 +0800

PA2 stage3 finished. PA2 all stages cleared. Implemented 3 ioe functions to
support the I/O ability of NEMU. Implemented clock. Implemented keyboard detecti
ng. Now you can run the 2 mini-games in /nexus-am/apps (dir).

commit 71635751ad4f61a8bae2fc051dc48d604e9c6585
Author: tracer-ics2017 <tracer@njuics.org>
Date: Sun Apr 26 19:41:10 2026 +0800

> run
2311887
illum
Linux ubuntu 4.15.0-142-generic #146~16.04.1-Ubuntu SMP Tue Apr 13 09:26:57
UTC 2021 i686 i686 i686 GNU/Linux
19:41:10 up 5:47, 1 user, load average: 0.96, 1.03, 0.85
5ec710cd4aa2652249a84c923ef32bb315eaabb

commit 469c3c022f6585316038a2f6a535dc5ff0402a30
Author: tracer-ics2017 <tracer@njuics.org>
Date: Sun Apr 26 19:38:42 2026 +0800
:
```

图 28: 手动 git 记录

至此 PA 2 代码实现全部完成

四、 实验环境说明

由于整个课程的实验部分没有要求提交 PA0 / 环境配置一节的报告，此处对实验环境的搭建作简单说明。

(一) 虚拟机配置

采用 VMware 15.5.0 + Ubuntu 16.04 ([ubuntu-16.04.4-desktop-i386.iso](#)) 有可视化界面的虚拟机环境。

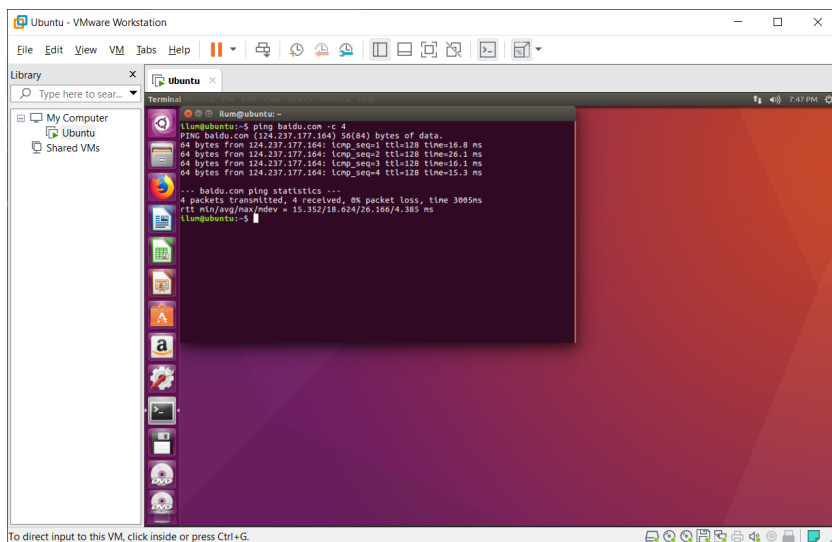


图 29: VMWare + Ubuntu

网络连接无问题。

按照指导书要求，使用 `sudo apt-get install` 命令下载并安装相应的资源工具包。

(二) IDE 配置

此前编程习惯为 vscode 的 IDE 环境，故尝试将此 Ubuntu 虚拟机连接到 vscode。

此前尝试 vscode 的 SSH 功能、VMware 自带的共享文件夹，均失败。

最终使用 SSHFS 的方案，把 Ubuntu 的目录通过 SSHFS 作为一个附加卷挂载到宿主 Windows。安装 WinFsp 和 SSHFS-Win 工具，而后只需要在 Windows cmd 输入命令 `net use X: \\sshfs\[username]@[ip.addr]\[password] /user:[username]`（方括号需要替换为相应内容）在宿主机的文件系统根目录形成一个新的卷轴“X:”，其中内容就是对 Ubuntu 虚拟机指定文件夹的挂载。在 X 盘内右键用 vscode 打开即可用 IDE 环境来编辑代码，同步生效于 Ubuntu 虚拟机。

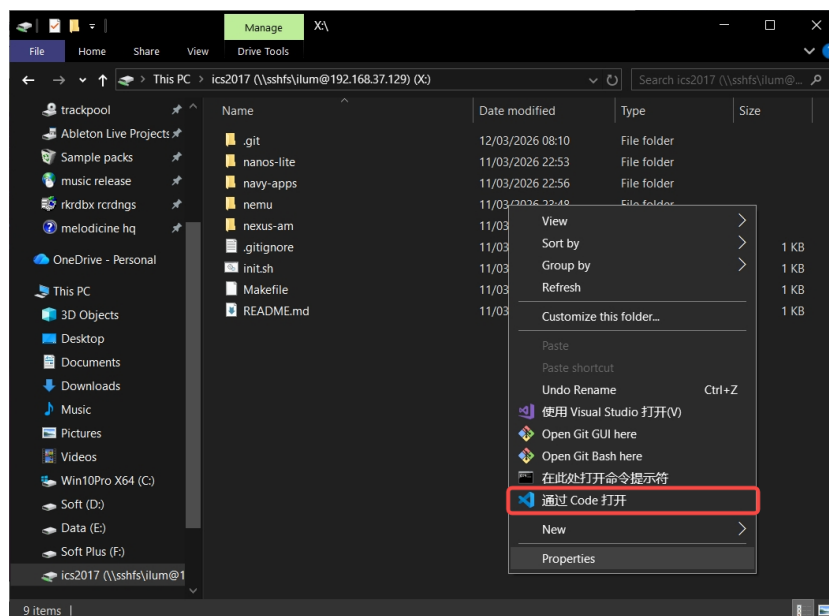


图 30: 本地 vscode 编写代码

五、 总结

PA2 主要分为实现 CPU 指令和简单的 I/O 接口两方面。CPU 指令的实现，让我首次对于 RTL（寄存器传输语言）有了代码层面的了解，在 RTL 指令/伪指令的函数补全过程中也让我对基础寄存器的构成和标志位寄存器印象更加深刻了。I/O 接口实现方面，这是全新的体验，初步理解了输入输出信号如何在 CPU 内部和外界 I/O 设备之间传输。运行一些结合了 CPU 指令和 I/O 而实现的小程序，使得实验更具激发性。

此外，在实验中，原代码框架大量使用宏定义做函数展开的做法很有趣，让我理解到 C 语言中想要批量生成结构很相似、名字有规律的函数，用宏是最方便的。

在必答题和思考题的解答过程中，进一步学习到一些原理性知识，特别是 Makefile 和 C 语言底层操作相关的。volatile 关键字的知识是全新的，让我对于程序优化与语义完整之间的平衡有了全新的认识。

PA2 实验代码已同步备份至 GitHub 仓库。